

From Local Summaries to Global Certificates: Predictive Memory Sizing for Distributed Heavy-Hitter Tracking

Nikolaos Chalvantzis
National Technical University of Athens
Athens, Greece
nchalv@cslab.ece.ntua.gr

ABSTRACT

Modern distributed stream processing systems rely on partitioning to scale, but workload skew can severely degrade performance by overloading downstream workers. Identifying global heavy hitters is essential for mitigation, yet it remains a challenging coordinator-side inference problem when raw records cannot be centralized and local partition memory is strictly bounded. In this paper, we propose a unified framework for distributed heavy-hitter tracking that bridges deterministic inference and adaptive memory provisioning. First, we introduce a routing-agnostic certification layer that converts bounded local Space-Saving telemetry into deterministic global frequency intervals. By explicitly decomposing coordinator error into reporter inflation and hidden non-reporter mass, we establish a strict discoverability floor that guarantees no global heavy hitter is missed. Second, we build a capacity controller on top of these certificates. An observed margin violation activates an origin-aware upward response, whereas memory is released only through measured downward probes because high-capacity telemetry cannot certify an unobserved smaller summary. A residual guard prevents a failed probe from being repeated aggressively, while a separate materiality-cost guard raises memory only when doing so resolves a worthwhile fraction of threshold-crossing ambiguity. We also show how the same certification and planning logic extends to a hybrid exact-approximate regime that reserves exact state for stable dominant keys while tracking the residual tail approximately. Extensive evaluations demonstrate that our framework provides robust deterministic guarantees while optimizing the memory-quality tradeoff across highly volatile streams.

PVLDB Reference Format:

Nikolaos Chalvantzis. From Local Summaries to Global Certificates: Predictive Memory Sizing for Distributed Heavy-Hitter Tracking. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Modern distributed stream processing systems rely on input partitioning to scale keyed operators such as aggregations, joins, and stateful enrichments. This architecture is effective when the input is reasonably balanced, but it becomes fragile under skew, where a small number of keys accounts for a disproportionate share of the stream. Such keys can overload downstream workers, degrade latency, and undermine the effectiveness of otherwise well-provisioned execution pipelines. In many deployments, the ability to identify these dominant keys early is a prerequisite for mitigation mechanisms such as selective replication, routing overrides, or workload-aware state placement.

A natural object of interest is the set of *global heavy hitters*: keys whose global relative frequency exceeds the balancing threshold induced by the number of downstream workers. Detecting such keys in a distributed stream, however, is fundamentally a coordinator-side inference problem. Each input partition observes only a fragment of the window, per-partition memory is bounded, and centralizing all raw records at the barrier is infeasible. A key that is globally important may appear only weakly, or not at all, in many local views. If it is omitted from all local summaries, it is unrecoverable. Thus, sketch sizing is not merely a question of compression efficiency; it is directly tied to discoverability and correctness.

This raises two closely related questions. First, given only compact local telemetry, what can a coordinator certify *deterministically* about global heavy hitters in a realized window? Second, how should local sketch capacity be chosen for the *next* window so that these guarantees remain useful under changing workloads? Existing sketching primitives provide local retention and frequency bounds, but the coordinator still needs a routing-agnostic way to turn those local guarantees into global certified conclusions. Likewise, adaptive provisioning should ideally be driven by a quantity that is directly meaningful for the actuator, rather than by an indirect notion of difficulty.

This paper addresses both questions through a unified framework built around local Space-Saving summaries. Our starting point is a deterministic coordinator-side certification layer: from local Space-Saving telemetry, the coordinator constructs a global candidate set, derives certified frequency intervals for reported keys, and partitions them into certified heavy hitters, certified non-heavy-hitters, and ambiguous keys. These guarantees are computed from the realized window only and require no stochastic routing assumptions. The key technical device is a signed decomposition of coordinator error into two components: *reporter inflation*, caused by one-sided local overestimation, and *hidden non-reporter mass*, caused by partitions that do not report a key. This decomposition

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

yields explicit routing-agnostic envelopes and a simple discoverability floor under which no true heavy hitter can be missed at the candidate stage.

We then build the adaptive layer on top of these deterministic certificates. For each realized window, the coordinator tests whether the deployed capacity met a prescribed deterministic margin over a chosen service fraction of HH-relevant candidates. A violation is converted directly into an upward capacity demand using the capacity at which the margin was observed. A successful window does not produce a smaller counterfactual target: ordinary summary telemetry cannot determine which keys and certificates would have survived at an unobserved lower capacity. The controller therefore separates evidence-backed upward sizing from experimental downward release.

The result is a two-layer architecture. The first layer provides deterministic coordinator-side guarantees for a realized window. The second uses the output of the first to plan memory for the next window. This separation is important both conceptually and operationally. It ensures that correctness claims about the realized window do not depend on controller success, while still allowing the system to adapt its memory footprint over time. The service-margin test and upward demand provide the primary control evidence, while unresolved threshold-crossing certificates act as a conservative guard when distributed evidence is not sharp enough at the requested margin scale.

Our contributions are as follows:

- We develop a deterministic coordinator-side certification framework that converts local Space-Saving telemetry into routing-agnostic global frequency intervals, together with certified heavy hitters, certified non-heavy-hitters, and ambiguous keys.
- We establish a simple discoverability guarantee: above a natural per-partition size floor, every global heavy hitter is guaranteed to appear in the coordinator's candidate set, eliminating false negatives at the discovery stage.
- We introduce a one-sided formulation of adaptive sizing. Margin violations produce an origin-aware upward capacity demand, while successful observations certify only the deployed capacity.
- We combine immediate upward response with residual-guarded downward probes, then extend the controller with ambiguity-aware and hybrid tracking.

The rest of the paper is organized as follows. Section 2 formalizes the system model and coordinator interface. Section 3 presents the local Space-Saving primitive and the telemetry contract. Section 4 develops deterministic coordinator-side certification for global heavy hitters. Section 5 converts realized margins into one-sided capacity pressure. Section 6 presents the next-window controller. Section 7 extends the framework to hybrid exact-approximate tracking, and Section 8 outlines the experimental evaluation.

2 SYSTEM MODEL AND COORDINATOR INTERFACE

We consider a windowed distributed stream processing system that ingests a high-rate key-value stream and executes keyed operators over n parallel downstream workers. Input arrives through m disjoint input partitions. These two roles are deliberately separated. Input partitions observe local fragments of the stream and emit

compact telemetry at window boundaries, while downstream workers are the entities whose load balance motivates the heavy-hitter threshold. The setting is representative of modern streaming deployments in which raw records cannot be centralized continuously, yet coordinator-side decisions must still be made from bounded summary state.

Processing proceeds in fixed, non-overlapping windows indexed by $t \in \mathbb{N}$. Let $\mathcal{W}^t$ denote the multiset of all records arriving in window t , and let $N^t = |\mathcal{W}^t|$ be its total mass. Partition $i \in [m]$ observes a local substream $\mathcal{W}_i^t$ with mass $N_i^t = |\mathcal{W}_i^t|$, where $\sum_{i=1}^m N_i^t = N^t$. We make no assumptions about how the stream is split across partitions: the system may experience arbitrary imbalance both across partitions and across windows. Each record carries a key $k \in \mathcal{K}$, where $\mathcal{K}$ is the key universe. We analyze nonempty windows $N^t > 0$; an empty global window contains no heavy hitters and all certification statements are vacuous. Let

$$\mathcal{A}^t := \{i \in [m] : N_i^t > 0\}$$

denote the set of active partitions in window t . We write $f^t(k)$ and $f_i^t(k)$ for the global and local counts of key k , and we write

$$p^t(k) := \frac{f^t(k)}{N^t}, \quad p_i^t(k) := \frac{f_i^t(k)}{N_i^t}$$

for the corresponding global relative frequency and, when $i \in \mathcal{A}^t$, local relative frequency. For inactive partitions $i \notin \mathcal{A}^t$, all local counts are zero and $p_i^t(k)$ is not used; their summaries contribute only zero mass and zero minimum-counter terms under the conventions below. Unless ambiguity would arise, we suppress the superscript t when the reference window is clear.

The paper is concerned with *global heavy hitters*, that is, keys whose global frequency exceeds the natural balancing threshold induced by the number of downstream workers.

DEFINITION 1 (GLOBAL HEAVY HITTER). A key $k \in \mathcal{K}$ is a *global heavy hitter in window t* if

$$p^t(k) > \frac{1}{n}.$$

We denote the set of global heavy hitters by

$$\mathcal{H}_n^t := \left\{k \in \mathcal{K} : p^t(k) > \frac{1}{n}\right\}.$$

This threshold is operationally meaningful: a key whose mass exceeds $1/n$ cannot be evenly absorbed by n downstream workers without some form of special treatment. This is the canonical balancing threshold for keyed downstream execution over n workers. More complex downstream operators may induce different operational thresholds, but such cases are outside the scope of the present paper. The heavy-hitter set is also small in a worst-case sense.

PROPOSITION 1 (CARDINALITY BOUND). For every window t ,

$$|\mathcal{H}_n^t| < n.$$

PROOF. If $\{k_1, \dots, k_r\} \subseteq \mathcal{K}$ and $p^t(k_j) > 1/n$ for all j , then

$$1 = \sum_{k \in \mathcal{K}} p^t(k) \geq \sum_{j=1}^r p^t(k_j) > \frac{r}{n},$$

hence $r < n$. $\square$

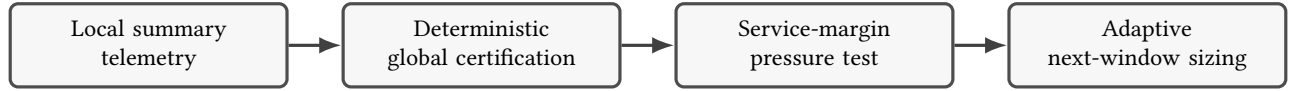


Figure 1: Conceptual pipeline of the proposed framework. The coordinator converts bounded local summary telemetry into deterministic global certificates, tests the observed service margin, and combines upward response with measured downward probing for the next window.

This bound is purely combinatorial and holds regardless of skew or partitioning. In realistic workloads, the number of global heavy hitters is often much smaller than n , but the worst-case bound provides a useful scale for certification and provisioning.

The coordinator’s per-window task has two parts. First, it must ensure *discoverability*: every global heavy hitter should appear in at least one local report, so that it is not lost before global inference even begins. Second, for every reported key, it should derive deterministic frequency bounds strong enough to support certified heavy-hitter decisions.

To support this inference task, each input partition maintains a bounded local summary over its local window and emits only compact telemetry at the barrier. The main regime studied in the paper uses a single fixed-size summary per partition; the specific summary primitive is introduced in Section 3. At this level of abstraction, what matters is that partition i exports its local window mass N_i^t , a bounded dictionary of retained keys with summary-specific metadata, and a small amount of scalar telemetry that allows the coordinator to reason about keys that were *not* retained locally. The coordinator never sees raw records. Its view of the window is entirely mediated by this telemetry contract.

From the summaries received at the barrier, the coordinator forms a global candidate set $\mathcal{U}^t$, consisting of all keys reported by at least one partition. It then computes, for each $k \in \mathcal{U}^t$, a deterministic interval

$$[\underline{p}^t(k), \bar{p}^t(k)]$$

that encloses the true global frequency $p^t(k)$. These intervals support a three-way partition of the candidate set into certified heavy hitters, certified non-heavy-hitters, and ambiguous keys. The concrete construction of $\mathcal{U}^t$ and of these deterministic envelopes depends on the local summary primitive and is developed later; for now, the important point is that coordinator-side inference is driven entirely by bounded telemetry emitted at the barrier.

A central organizational principle of the paper is the separation between *realized-window certification* and *next-window planning*. Once a window has completed and all local telemetry has been received, the coordinator-side guarantees derived in this paper are deterministic. They do not rely on routing randomness, temporal smoothness, or workload stationarity. By contrast, choosing memory for the next window is inherently predictive: it requires extrapolating from what was observed in window t to what may happen in window $t + 1$. This distinction is not merely expository. It is what allows the paper to make strong correctness claims about the realized window while treating adaptation as a separate, explicitly predictive layer. If the system mis-sizes a future window, the realized-window certificates remain valid; only the sharpness of the next round of inference is affected.

The output of the realized-window layer is therefore concrete and finite. For each window t , the coordinator returns the candidate set $\mathcal{U}^t$, deterministic frequency intervals for all candidates, and a partition of candidates into certified positives, certified negatives, and ambiguous keys. Together, these objects form the coordinator-side certificate for the realized window. The planning layer then uses those same certificates to infer a memory-valued requirement estimate for the realized window, and to convert that estimate into a sizing decision for the next window. Figure 1 summarizes this architecture.

Table 1 summarizes the main notation used throughout the paper.

The remainder of the paper develops this pipeline in stages. Section 3 introduces the local summary primitive and the telemetry it exposes. Section 4 shows how the coordinator converts that telemetry into deterministic global certificates. Section 5 turns those certificates into a service-margin test and one-sided upward demand, and Section 6 combines that evidence with downward probing. A hybrid exact–approximate regime is treated as an extension rather than as part of the core theorem path.

3 LOCAL SPACE-SAVING SUMMARIES AND TELEMETRY

This section introduces the local summary primitive used throughout the paper and isolates the properties that will later support coordinator-side certification. Our main regime assigns one bounded summary to each input partition in each window. The purpose of the summary is not merely to compress local frequency information, but to expose enough structure at the barrier for a coordinator to reason deterministically about the global stream.

We use *Space-Saving* (SS) as the local primitive, implemented faithfully via the original *Stream-Summary* data structure of Metwally et al. [17]. In our setting, each partition maintains one such summary per window and exports its end-of-window telemetry at the barrier. Space-Saving is well suited to this role for three reasons. First, it retains explicitly named keys rather than only hashed counters, which makes coordinator-side aggregation and certification possible without an additional key-recovery mechanism. Second, it provides a deterministic local retention threshold: any key whose local relative frequency exceeds $1/q$ must appear in a summary of size q . Third, every retained key is accompanied by a one-sided frequency certificate, while the minimum counter in the final summary gives a direct bound on the mass of any key that was not retained locally. These are precisely the ingredients needed in the next section to separate coordinator error into inflation from reporting partitions and hidden mass from non-reporting ones.

Table 1: Main notation. Time superscripts are omitted when the reference window is clear from context.

Symbol	Meaning
<i>System and stream model</i>	
n, m	Number of downstream workers (n) and input partitions (m)
$\mathcal{K}$	Universe of possible keys
$\mathcal{W}, \mathcal{W}_i$	Global window multiset and partition- i local substream
$N, N_i, \mathcal{A}$	Global window mass, partition-local mass, and active partitions $\{i : N_i > 0\}$
$f(k), f_i(k)$	Global and local counts of key k
$p(k), p_i(k)$	Global and local relative frequencies of key k
$\mathcal{H}_n$	Global heavy-hitter set $\{k : p(k) > 1/n\}$
<i>Local summaries and coordinator observables</i>	
$\mathcal{S}_i, q$	Partition- i Space-Saving summary and its common local capacity
$\hat{f}_i(k), \varepsilon_i(k)$	Local estimate of key k and its one-sided overestimation certificate
c_i	Final minimum-counter bound of partition i ; also the bound used for hidden non-reporter mass
$\mathcal{U}$	Union of keys reported by at least one partition in the realized window
$R(k)$	Set of partitions whose local summaries report key k
$\hat{f}(k), \hat{p}(k)$	Coordinator aggregate count estimate and corresponding relative-frequency estimate
<i>Deterministic certification</i>	
$\underline{p}(k), \bar{p}(k)$	Deterministic lower and upper bounds on the true global relative frequency $p(k)$
$K_+, K_-, K?$	Certified heavy hitters, certified non-heavy-hitters, and ambiguous candidates
T	HH-relevant target set $K_+ \cup K?$ used by the planning layer
$I(k), H(k), M(k)$	Reporter-inflation term, hidden-mass term, and composite deterministic margin
<i>Planning and adaptive control</i>	
ε_M, r_M	Fractional and absolute deterministic margin budgets, with $r_M = \varepsilon_M/n$
α	Service fraction used to aggregate per-key deterministic margins
M_α	Window-level service margin $Q_\alpha(\{M(k) : k \in T\})$
$q_\uparrow$	Upward capacity demand activated only when $M_\alpha > r_M$
q_{probe}	Capacity used for a physical downward probe
$\bar{q}$	Baseline target selected by upward response or downward probing
L, g	Failed-probe lower boundary and temporary recovery guard
$s(k), A$	Per-key separation demand and selected ambiguity proposal
$\ell_{\text{Amb}}, q_{\text{Amb}}$	Observed and geometrically persisted ambiguity margins in log-capacity space
β	Geometric persistence factor for an unrefreshed ambiguity margin
q	Deployed common local summary size in the core regime
<i>Hybrid exact-approximate tracking</i>	
E, e, r, b	Promoted exact head, exact-head counters, residual-summary counters, and total hybrid local budget
$\mathcal{W}_{i,\text{res}}, \mathcal{S}_{i,\text{res}}$	Residual substream and residual Space-Saving summary at partition i
$N_{\text{res}}, N_{i,\text{res}}, \mathcal{A}_{\text{res}}$	Global residual mass, partition-local residual mass, and residual-active partitions
P_E	Fraction of global mass captured exactly by the promoted head E
$R_{\text{res}}(k), c_{i,\text{res}}$	Residual reporting set of key k and residual minimum-counter bound at partition i
$\underline{p}_{\text{res}}(k), \bar{p}_{\text{res}}(k)$	Deterministic residual frequency bounds for keys outside the exact head
r_{min}	Residual discoverability floor in the hybrid regime

3.1 Partition-Local Summary Primitive

Fix a window t and a partition i . The partition processes its local substream $\mathcal{W}_i^t$ using a Space-Saving summary $\mathcal{S}_i^t$ of capacity q^t . The summary stores at most q^t keys. For each retained key k , it maintains an estimated local count $\hat{f}_i^t(k)$ together with a nonnegative overestimation certificate $\varepsilon_i^t(k)$.

The update rule is standard. When an item with key k arrives, the summary increments the existing counter if k is already present. If k is absent and free capacity remains, the summary inserts k with unit count and zero error. Otherwise, the summary replaces the currently minimum counter entry: the evicted key is discarded, the incoming key takes its place, its new counter is set to one plus the minimum counter that was evicted, and that minimum counter is recorded as the new key's error certificate.

The implementation used in this paper is faithful to the original Space-Saving design of Metwally et al. Specifically, each local summary is realized as a *Stream-Summary*, which groups items of equal

Algorithm 1 Space-Saving update at partition i

Input: Summary $\mathcal{S}$ with capacity q , incoming key k

```

1: if  $k \in \mathcal{S}$  then
2:    $\hat{f}(k) \leftarrow \hat{f}(k) + 1$ 
3: else if  $|\mathcal{S}| < q$  then
4:   insert  $k$  with  $\hat{f}(k) \leftarrow 1$  and  $\varepsilon(k) \leftarrow 0$ 
5: else
6:    $x \leftarrow \arg \min_{u \in \mathcal{S}} \hat{f}(u)$ 
7:   replace  $x$  by  $k$ 
8:    $\hat{f}(k) \leftarrow \hat{f}(x) + 1$ 
9:    $\varepsilon(k) \leftarrow \hat{f}(x)$ 

```

estimated count into buckets maintained in sorted order. Monitored items are accessed directly through a hash table (or equivalent associative structure), while counter increments are handled by moving items between adjacent buckets as their estimated counts change. This representation supports constant-time lookup of monitored items together with lightweight local updates.

While the coordinator-side analysis depends only on the end-of-window invariants exposed by Stream-Summary, our local implementation remains faithful to the original Space-Saving design.

3.2 Summary Invariants

We now record the properties of Space-Saving that the coordinator will later use. For readability, we suppress the window superscript t in this subsection.

The first property is the familiar one-sided certificate for retained keys.

LEMMA 1 (ONE-SIDED LOCAL CERTIFICATE). *For any partition i and any key $k \in \text{keys}(\mathcal{S}_i)$,*

$$\hat{f}_i(k) \leq \hat{f}_i(k) \leq \hat{f}_i(k) + \varepsilon_i(k), \quad 0 \leq \varepsilon_i(k) \leq c_i \leq \frac{N_i}{q},$$

where

$$c_i := \begin{cases} 0, & |\text{keys}(\mathcal{S}_i)| < q, \\ \min_{x \in \text{keys}(\mathcal{S}_i)} \hat{f}_i(x), & |\text{keys}(\mathcal{S}_i)| = q. \end{cases}$$

Equivalently, c_i is the minimum over the q logical counters of the summary, treating unused counters as zero.

Thus the local estimate of a retained key never underestimates its true local count, and any overestimation is explicitly bounded. This one-sidedness is essential later: it ensures that partitions reporting a key can only contribute upward error to the coordinator.

The second property is the deterministic retention threshold.

LEMMA 2 (DETERMINISTIC LOCAL RETENTION). *For a partition i with $N_i > 0$, let $\tau = 1/q$. If a key k satisfies*

$$p_i(k) = \frac{\hat{f}_i(k)}{N_i} > \tau,$$

then k must belong to the retained key set $\text{keys}(\mathcal{S}_i)$.

This lemma is the basis of the discoverability result in Section 4. It converts a local support condition into a deterministic statement about visibility at the coordinator.

The third property concerns keys that are *not* present in the final local summary.

LEMMA 3 (ABSENCE BOUND VIA THE MINIMUM COUNTER). *For any partition i and any key $k \notin \text{keys}(\mathcal{S}_i)$,*

$$f_i(k) \leq c_i.$$

In particular,

$$c_i \leq \frac{N_i}{q}.$$

PROOF. If the summary is not full at the end of the window, no eviction has occurred, every key with positive local count is retained, and $c_i = 0$. Hence any absent key has $f_i(k) = 0$. If the summary is full, the first inequality is the standard Space-Saving absence invariant: any key that is not retained at the end can have accumulated no more mass than the final minimum counter. Otherwise, its local frequency would have forced it to displace some retained key. The second inequality follows because, in the full case, the q retained counters sum to N_i , so their minimum cannot exceed their average N_i/q . $\square$

This minimum counter plays a central role in the coordinator analysis. It does more than certify which keys were retained; it also gives the coordinator a uniform worst-case bound on the local mass of every key that was *not* retained. Thus c_i is precisely the scalar that later bounds the hidden non-reporter mass: if partition i does not report key k , then the coordinator may still charge up to c_i units of unseen local mass to k , but no more. As a result, non-reporting partitions remain visible to the coordinator through a scalar quantity, even when they do not explicitly mention a given key.

Finally, the update rule preserves an exact accounting identity that is useful for implementation sanity checks and for interpreting the summary as a partition-local compression of the full window.

LEMMA 4 (COUNTER-SUM IDENTITY). *At the end of the window,*

$$\sum_{x \in \text{keys}(\mathcal{S}_i)} \hat{f}_i(x) = N_i.$$

Every arriving item increments exactly one counter, so the sum of the retained counters equals the total local mass processed by the summary.

Taken together, Lemmas 1–3 provide the exact interface needed by the coordinator. Reporting partitions contribute explicit one-sided overestimation bounds, while non-reporting partitions contribute only bounded hidden mass through their minimum counters.

3.3 Barrier Telemetry

We treat end-of-window communication as an explicit telemetry contract between each input partition and the coordinator. The contract is window-local: partitions export only compact information derived from the realized window, and no temporal state is required for the deterministic certification layer.

At the end of window t , partition i transmits its local window mass N_i^t together with the contents of its Space-Saving summary,

$$\mathcal{S}_i^t = \left\{ (k, \hat{f}_i^t(k), \epsilon_i^t(k)) : k \text{ retained locally} \right\}, \quad |\mathcal{S}_i^t| \leq q^t.$$

The partition may also transmit the minimum counter

$$c_i^t := \begin{cases} 0, & |\mathcal{S}_i^t| < q^t, \\ \min_{x \in \text{keys}(\mathcal{S}_i^t)} \hat{f}_i^t(x), & |\mathcal{S}_i^t| = q^t, \end{cases}$$

which is the final minimum over the q^t logical counters, treating unused counters as zero. In practice, this value is derivable from the transmitted dictionary itself with a single $O(q^t)$ scan per partition at the barrier. Once computed, it is reused in all coordinator-side envelope calculations for that window. No raw local records are exported.

This telemetry is sufficient for all coordinator-side tasks in the core framework. From the union of locally retained keys, the coordinator forms a candidate set for the realized window. From the reported estimates and certificates, it computes deterministic lower and upper bounds for each candidate’s global frequency. From the per-partition minimum counters, it bounds the hidden mass contributed by partitions that do not report a candidate. The same telemetry therefore supports both discoverability arguments and quantitative frequency envelopes.

It is important to emphasize what the coordinator is *not* assumed to do. The coordinator is not required to merge local Space-Saving summaries into a new valid Space-Saving instance for continued streaming use. In the present paper it acts as a terminal sink for window-level inference: it consumes local telemetry, produces deterministic certificates for the realized window, and emits planning signals for the next one. This choice keeps the reasoning faithful to the barriered streaming setting and avoids introducing an unnecessary mergeability requirement into the theorem path.

Section 2 defined the system model and the coordinator interface at a high level. The present section makes that interface concrete by identifying the local primitive and the invariants it exposes. The next section uses these invariants to derive the main deterministic results of the paper: a discoverability floor for global heavy hitters, a signed decomposition of coordinator-side error, and explicit routing-agnostic frequency intervals for all reported candidates.

4 DETERMINISTIC GLOBAL HEAVY-HITTER CERTIFICATION

We now turn to the main inference problem of the paper. At the end of a realized window, the coordinator receives bounded telemetry from all partitions and must reason about global heavy hitters without access to raw records. The goal of this section is to show that the local invariants of Section 3 are already strong enough to support routing-agnostic global certification. Specifically, we establish three results. First, above a simple size floor, every global heavy hitter is guaranteed to appear in the coordinator’s candidate set. Second, for every reported candidate, the coordinator can derive an explicit deterministic interval for its true global relative frequency. Third, these intervals induce a sound partition of candidates into certified heavy hitters, certified non-heavy-hitters, and ambiguous keys.

4.1 Coordinator Aggregation

Fix a realized window t . Each partition $i \in [m]$ exports its local mass N_i^t , its Space-Saving summary S_i^t , and the associated end-of-window certificates described in Section 3. The coordinator begins by forming the union of all locally reported keys:

$$\mathcal{U}^t := \bigcup_{i=1}^m \text{keys}(S_i^t).$$

This is the candidate set for the realized window. Every key outside $\mathcal{U}^t$ is absent from every local summary and must therefore be handled only indirectly through the minimum-counter bounds of non-reporting partitions.

For any candidate $k \in \mathcal{U}^t$, define its reporting set by

$$R^t(k) := \{i \in [m] : k \in \text{keys}(S_i^t)\}.$$

The coordinator's natural aggregate estimate of the global count of k is then

$$\hat{f}^t(k) := \sum_{i \in R^t(k)} \hat{f}_i^t(k), \quad \hat{p}^t(k) := \frac{\hat{f}^t(k)}{N^t}, \quad N^t := \sum_{i=1}^m N_i^t.$$

This aggregate uses only the partitions that explicitly report k . The key question is therefore how to account for the partitions in $[m] \setminus R^t(k)$, which do not report k but may still contain some of its mass.

4.2 Discoverability Guarantee

We first establish the candidate-discovery guarantee. The coordinator can only certify a heavy hitter once that key appears in $\mathcal{U}^t$, so the first concern is whether a true heavy hitter can be absent from every local summary.

PROPOSITION 2 (DISCOVERABILITY FLOOR). *If the common local summary size in window t satisfies*

$$q^t \geq n,$$

then every global heavy hitter $k \in \mathcal{H}_n^t$ appears in the candidate set $\mathcal{U}^t$.

PROOF. Let k be a global heavy hitter, so that $p^t(k) > 1/n$. Suppose, for contradiction, that

$$p_i^t(k) \leq \frac{1}{n} \quad \text{for all } i \in \mathcal{A}^t.$$

Inactive partitions have $f_i^t(k) = 0$, so

$$f^t(k) = \sum_{i=1}^m f_i^t(k) \leq \sum_{i=1}^m \frac{N_i^t}{n} = \frac{N^t}{n},$$

which implies $p^t(k) \leq 1/n$, a contradiction. Hence there exists some active partition $i \in \mathcal{A}^t$ such that

$$p_i^t(k) > \frac{1}{n}.$$

If $q^t \geq n$, then $1/q^t \leq 1/n$, and therefore

$$p_i^t(k) > \frac{1}{q^t}.$$

By Lemma 2, k must belong to $\text{keys}(S_i^t)$, and thus $k \in \mathcal{U}^t$. $\square$

This proposition is the coordinator-side analogue of the local retention threshold. It says that once the local summary size reaches the balancing threshold n , the discovery stage itself introduces no false negatives for global heavy hitters. This floor is tight for the worst-case visibility pattern: if a global heavy hitter distributes its mass as evenly as possible across partitions, then the most informative partition may see it at local relative frequency only slightly above $1/n$, so the local retention threshold $1/q^t$ must not exceed $1/n$, which is exactly the condition $q^t \geq n$. The condition is therefore sufficient, not necessary, for discoverability; in many realized windows, a global heavy hitter is much more concentrated than this worst case and remains discoverable even under smaller local summaries.

4.3 Deterministic Frequency Envelopes

We now derive deterministic frequency intervals for the candidates in $\mathcal{U}^t$. The central observation is that coordinator-side error has a natural signed decomposition. For a candidate $k \in \mathcal{U}^t$,

$$\hat{f}^t(k) - f^t(k) = \sum_{i \in R^t(k)} (\hat{f}_i^t(k) - f_i^t(k)) - \sum_{j \notin R^t(k)} f_j^t(k).$$

The first term is the upward error contributed by reporting partitions. The second term is the hidden mass contributed by non-reporting partitions. These two terms are controlled by different summary invariants.

For reporting partitions, Lemma 1 gives

$$0 \leq \hat{f}_i^t(k) - f_i^t(k) \leq \varepsilon_i^t(k).$$

For non-reporting partitions, Lemma 3 gives

$$f_j^t(k) \leq c_j^t,$$

where c_j^t is the transmitted minimum-counter bound defined in Section 3.3. Combining these bounds yields the following deterministic envelope.

PROPOSITION 3 (DETERMINISTIC GLOBAL ENVELOPE). *For every candidate $k \in \mathcal{U}^t$,*

$$- \sum_{j \notin R^t(k)} c_j^t \leq \hat{f}^t(k) - f^t(k) \leq \sum_{i \in R^t(k)} \varepsilon_i^t(k).$$

Equivalently,

$$\hat{f}^t(k) - \sum_{i \in R^t(k)} \varepsilon_i^t(k) \leq f^t(k) \leq \hat{f}^t(k) + \sum_{j \notin R^t(k)} c_j^t.$$

PROOF. The result follows directly from the signed decomposition above together with the one-sided local certificates for $i \in R^t(k)$ and the minimum-counter absence bounds for $j \notin R^t(k)$. $\square$

The upper bound intentionally contains no subtraction for reporting partitions. For a reporting partition $i \in R^t(k)$, the true local count $f_i^t(k)$ may be anywhere between $\hat{f}_i^t(k) - \varepsilon_i^t(k)$ and $\hat{f}_i^t(k)$; when maximizing the global frequency, the worst case is zero reporter-side overcount, so the contribution is simply $\hat{f}_i^t(k)$. The only additional mass that can raise the global count beyond the aggregate reported estimate comes from partitions that did not report k , and Lemma 3 bounds each such hidden contribution by c_j^t .

After normalization by N^t , Proposition 3 yields an explicit deterministic interval for the global relative frequency of every candidate:

$$\underline{p}^t(k) := \max \left\{ 0, \hat{p}^t(k) - \frac{1}{N^t} \sum_{i \in R^t(k)} \varepsilon_i^t(k) \right\}, \quad (1)$$

$$\bar{p}^t(k) := \min \left\{ 1, \hat{p}^t(k) + \frac{1}{N^t} \sum_{j \notin R^t(k)} c_j^t \right\}. \quad (2)$$

COROLLARY 1 (DETERMINISTIC FREQUENCY INTERVAL). *For every candidate $k \in \mathcal{U}^t$,*

$$p^t(k) \in [\underline{p}^t(k), \bar{p}^t(k)].$$

The interval remains valid even when some partitions are empty or very small. Empty partitions have zero local mass and, by convention, zero minimum counter; small partitions contribute only their realized minimum-counter terms. The resulting bound may be loose, but it remains sound.

These intervals are the main quantitative output of the realized-window certification layer. They are routing-agnostic, depend only on the observed telemetry, and remain valid regardless of whether the next window is ultimately provisioned correctly or not.

4.4 Certification Partition

The intervals derived above support a three-way partition of the candidate set. Define

$$K_+^t := \{k \in \mathcal{U}^t : \underline{p}^t(k) > 1/n\}, \quad (3)$$

$$K_-^t := \{k \in \mathcal{U}^t : \bar{p}^t(k) \leq 1/n\}, \quad (4)$$

$$K_?^t := \mathcal{U}^t \setminus (K_+^t \cup K_-^t). \quad (5)$$

Thus K_+^t contains candidates that are certifiably heavy, K_-^t contains candidates that are certifiably non-heavy, and $K_?^t$ contains those whose true status remains unresolved by the available telemetry.

PROPOSITION 4 (SOUNDNESS OF THE CERTIFICATION PARTITION). *For every realized window t :*

- (1) *if $k \in K_+^t$, then $p^t(k) > 1/n$;*
- (2) *if $k \in K_-^t$, then $p^t(k) \leq 1/n$.*

PROOF. If $k \in K_+^t$, then $\underline{p}^t(k) > 1/n$. Since $p^t(k) \geq \underline{p}^t(k)$ by Corollary 1, it follows that $p^t(k) > 1/n$. Similarly, if $k \in K_-^t$, then $\bar{p}^t(k) \leq 1/n$, and since $p^t(k) \leq \bar{p}^t(k)$, we obtain $p^t(k) \leq 1/n$. $\square$

The partition is therefore sound on both sides: certified positives are true heavy hitters, and certified negatives are true non-heavy-hitters. Ambiguity is the only place where the telemetry leaves room for uncertainty.

When the discoverability floor of Proposition 2 is enforced, this candidate-side partition lifts to a statement about the full heavy-hitter set.

COROLLARY 2 (NO FALSE NEGATIVES UNDER THE DISCOVERABILITY FLOOR). *If $q^t \geq n$, then every global heavy hitter belongs to $K_+^t \cup K_?^t$.*

PROOF. By Proposition 2, every global heavy hitter belongs to $\mathcal{U}^t$. By Proposition 4, no true heavy hitter can belong to K_-^t . Hence every heavy hitter in $\mathcal{U}^t$ must lie in $K_+^t \cup K_?^t$. $\square$

This is the exact form of certification that the later planning layer will use: the realized window yields a set of certifiably heavy keys, a set of certifiably irrelevant ones, and a residual frontier of unresolved candidates.

Section 2 introduced the architectural separation between realized-window certification and next-window planning. The present section completes the first half of that architecture. From bounded local telemetry alone, the coordinator can now (i) guarantee discovery of all global heavy hitters above a simple local size floor, (ii) derive deterministic intervals for every reported candidate, and (iii) partition the candidate set into certified positives, certified negatives, and unresolved keys.

These outputs are not only useful as end-of-window certificates. They also induce the right objects for adaptation. In particular, the next section aggregates the deterministic margins underlying $[\underline{p}^t(k), \bar{p}^t(k)]$ into a window-level service test. A failed test activates an upward capacity response. A passed test establishes only that the deployed capacity was sufficient; it does not infer an unobserved lower capacity.

5 CAPACITY PRESSURE FROM DETERMINISTIC CERTIFICATES

Section 4 established the deterministic inference layer of the framework. For each realized window, the coordinator obtains a candidate set, per-candidate frequency intervals, and a certification partition into heavy hitters, non-heavy-hitters, and ambiguous keys. The planning layer must now decide whether the capacity q^t that produced this telemetry was sufficient for a desired margin target.

This distinction is important. Telemetry collected at capacity q^t can demonstrate that q^t was insufficient, and it can quantify the observed shortfall. It cannot, in general, reconstruct what a smaller summary would have reported. Changing capacity changes retained keys, reporting sets, minimum counters, and overestimation certificates. We therefore use current telemetry only for a one-sided upward response. Memory reduction is treated separately as an experimental probing problem in Section 6.

5.1 Heavy-Hitter-Relevant Target Set

The coordinator does not need to provision memory uniformly for all reported keys. Keys that are already certified non-heavy-hitters are not operationally central to the heavy-hitter problem, whereas certified heavy hitters and unresolved near-threshold keys are exactly those that determine future decision quality. Accordingly, we define the heavy-hitter-relevant target set

$$T^t := K_+^t \cup K_?^t.$$

The role of T^t is purely organizational. It identifies the portion of the candidate set on which the next-window sizing policy should focus. Certified positives in K_+^t should continue to be tracked accurately, while unresolved keys in $K_?^t$ are precisely those whose status may change or become certifiable under improved telemetry. By contrast, keys in K_-^t have already been shown to lie safely below the heavy-hitter threshold in the realized window and therefore do not need to dominate the sizing decision.

5.2 Deterministic Per-Key Margins

The interval construction of Section 4 already decomposes coordinator-side uncertainty into two interpretable components. For each candidate $k \in \mathcal{U}^t$, define

$$I^t(k) := \frac{1}{N^t} \sum_{i \in R^t(k)} \varepsilon_i^t(k), \quad (6)$$

$$H^t(k) := \frac{1}{N^t} \sum_{j \notin R^t(k)} c_j^t, \quad (7)$$

where $I^t(k)$ is the contribution of reporting partitions through one-sided overestimation, and $H^t(k)$ is the hidden-mass contribution of non-reporting partitions. In terms of these quantities,

$$\underline{p}^t(k) = \max\{0, \hat{p}^t(k) - I^t(k)\}, \quad \bar{p}^t(k) = \min\{1, \hat{p}^t(k) + H^t(k)\}.$$

The natural deterministic error scale for candidate k is therefore

$$M^t(k) := \max\{I^t(k), H^t(k)\}. \quad (8)$$

This is the smallest symmetric radius that dominates both the lower-side and upper-side uncertainty contributions induced by the certification envelopes. In particular, for every $k \in \mathcal{U}^t$,

$$|\hat{p}^t(k) - p^t(k)| \leq M^t(k).$$

The quantity $M^t(k)$ tells us whether candidate k 's realized telemetry supports the desired quality target. The next step aggregates this evidence across the heavy-hitter-relevant frontier.

5.3 Window-Level Service Margin

Fix a threshold-normalized deterministic margin budget $\varepsilon_M > 0$. Intuitively, ε_M specifies how much deterministic certificate slack is acceptable as a fraction of the heavy-hitter threshold $1/n$. It induces the absolute per-key margin budget

$$r_M := \frac{\varepsilon_M}{n}.$$

For example, if $n = 200$, the global heavy-hitter threshold is $1/n = 0.005$. Choosing $\varepsilon_M = 0.2$ sets $r_M = 0.001$, so the allowed deterministic margin is one fifth of the threshold.

Fix a service fraction $\alpha \in (0, 1]$. The window-level service margin is

$$M_\alpha^t := \begin{cases} 0, & T^t = \emptyset, \\ Q_\alpha(\{M^t(k) : k \in T^t\}), & T^t \neq \emptyset. \end{cases} \quad (9)$$

Thus α controls how broadly the margin target is enforced across the heavy-hitter frontier. For example, $\alpha = 0.95$ tests the margin attained by the lower 95% of HH-relevant candidates rather than allowing the hardest outlier to determine every decision.

The observation has two possible meanings:

$$\begin{aligned} M_\alpha^t > r_M &\implies q^t \text{ is insufficient,} \\ M_\alpha^t \leq r_M &\implies q^t \text{ is sufficient.} \end{aligned} \quad (10)$$

The second statement applies only to the deployed capacity. It does not imply that any $q < q^t$, including the floor n , would have passed the same test.

5.4 Upward Capacity Demand

When $M_\alpha^t > r_M$, current telemetry has demonstrated insufficiency. Reporter inflation and hidden non-reporter mass are controlled by Space-Saving terms whose worst-case scale is proportional to $1/q$. For upward response only, we therefore use the local model

$$M_\alpha(q) \approx \frac{C_\alpha^t}{q}, \quad \widehat{C}_\alpha^t := q^t M_\alpha^t.$$

Inverting this model gives the upward capacity demand

$$q_\uparrow^t := \begin{cases} \max\left\{q^t + 1, \left\lceil q^t \frac{M_\alpha^t}{r_M} \right\rceil\right\}, & M_\alpha^t > r_M, \\ \perp, & M_\alpha^t \leq r_M. \end{cases} \quad (11)$$

The origin capacity q^t is essential: it identifies the operating point at which M_α^t was measured. The estimate $q_\uparrow^t$ is not an exact counterfactual optimum, because increasing capacity can still change the candidate and reporting sets. It is, however, directionally supported by an observed violation. No analogous downward inversion is made after a successful window.

REMARK 1 (ONE-SIDED PRESSURE SIGNAL). Equation (11) is deliberately asymmetric. A violation produces a memory-valued upward demand. A successful observation produces only the statement that q^t was sufficient. Assigning a smaller capacity by applying the same inverse model downward would treat unavailable counterfactual telemetry as if it had been observed.

5.5 Interpretation of the Pressure Signal

The planning signal now has a precise interpretation. If the service margin violates its budget, $q_\uparrow^t$ estimates the larger capacity suggested by the inverse envelope model at the observed origin q^t . If the budget is met, the coordinator learns no smaller target. This prevents a well-provisioned window from manufacturing an unsupported recommendation to return to $q = n$.

The remaining question is therefore not how to forecast a bidirectional required-capacity sequence. It is how to combine immediate upward response with cautious experimental release. Section 6 addresses this by physically probing lower capacities only after repeated sufficiency, using each probe's own telemetry to accept the reduction or compute a fresh upward response.

6 ADAPTIVE NEXT-WINDOW SIZING

Section 5 defined a window-level service margin M_α^t and a one-sided upward demand $q_\uparrow^t$. This section completes the controller. Demonstrated insufficiency activates the upward response immediately. Demonstrated sufficiency holds the current capacity and may eventually authorize a physical downward probe. Threshold-crossing ambiguity is handled separately through an observable capacity frontier.

Capacity notation. The controller has a simple three-stage flow. The symbol q^t is the capacity actually deployed in window t . Temporal control produces a baseline proposal $\bar{q}^{t+1}$, using either the upward proposal $q_\uparrow^t$, the downward proposal q_{probe}^{t+1} , or the current capacity. Ambiguity may then raise the baseline, producing the final deployed capacity q^{t+1} . All other symbols in this section describe

evidence or temporary guard state rather than additional deployed capacities.

6.1 Capacity-Relative Evidence

The certificates used to construct M_α^t were produced at the deployed capacity q^t . The observation therefore follows the feedback path

$$q^t \longrightarrow \text{certificate geometry in window } t \longrightarrow M_\alpha^t.$$

Window t is *insufficient* when

$$M_\alpha^t > r_M, \quad (12)$$

and *sufficient* otherwise. In the insufficient case, Equation (11) supplies $q_\uparrow^t$ as the next baseline target. In the sufficient case, q^t becomes an observed sufficient point, but no lower capacity is inferred. Ambiguity is deliberately excluded from this service test, so extra memory deployed by the ambiguity guard cannot be misinterpreted as evidence about baseline pressure.

6.2 Margin-Guided Downward Probing and Residual Safety

Because ordinary summary telemetry cannot certify an unobserved smaller capacity, memory release requires a physical probe. A sufficient observation nevertheless indicates how much of the configured margin budget was consumed at the current operating point. Define the realized pressure ratio

$$\rho^t := \frac{M_\alpha^t}{r_M}. \quad (13)$$

Values above one trigger the upward response of Equation (11). When $0 \leq \rho^t \leq 1$, the deployed capacity is sufficient, but ρ^t is still capacity-relative and cannot certify a smaller summary. We therefore use it only to control the depth of an explicit probe.

After two consecutive sufficient observations, the controller uses $\max\{n, \lceil q^t \rho^t \rceil\}$ as a provisional pressure-scaled boundary. This boundary is not asserted to be sufficient. The controller moves only halfway toward it in linear capacity space and limits any single release to approximately $n/2$:

$$q_{\text{probe}}^{t+1} := \max\left\{n, q^t - \left\lfloor \frac{n}{2} \right\rfloor, \left\lceil \frac{q^t + \max\{n, \lceil q^t \rho^t \rceil\}}{2} \right\rceil\right\}. \quad (14)$$

Thus pressure affects how deeply the controller explores, not whether the unobserved lower capacity is declared safe. Even $\rho^t = 0$ authorizes only the bounded $n/2$ release because a zero observed margin may be censored by the deployed capacity.

If the probe is sufficient, the lower capacity is retained. If it is insufficient, the controller does not automatically rebound to a historical upper capacity. It uses the margin measured at the probe's origin capacity to compute a fresh upward response:

$$L^t := \max\{L^{t-1}, q_{\text{probe}}^t + 1\}, \quad \bar{q}^{t+1} := \max\{L^t, q_\uparrow^t\}. \quad (15)$$

This response distinguishes an overly aggressive probe from a genuine upward regime change without assuming that an earlier sufficient capacity remains the correct target.

Collecting the cases, the baseline actuation target is

$$\bar{q}^{t+1} := \begin{cases} \max\{L^t, q_\uparrow^t\}, & M_\alpha^t > r_M, \\ q_{\text{probe}}^{t+1}, & M_\alpha^t \leq r_M \text{ and a probe is authorized,} \\ q^t, & \text{otherwise.} \end{cases} \quad (16)$$

Thus the controller never lowers capacity merely because the current margin is small. Every reduction is an explicit probe.

After a failed probe, the residual guard retains the recovered demand

$$g^t := \max\{L^t, q_\uparrow^{t_{\text{fail}}}\}, \quad (17)$$

and is retained until the recovery capacity has produced two sufficient observations. During recovery the controller holds capacity at or above g^t , rather than immediately repeating a probe below the demand exposed by the failure. Once recovery is confirmed, the ordinary two-observation condition must be met again before another downward probe is issued.

This guarded margin-comfort policy is the canonical temporal controller. It reacts immediately to demonstrated insufficiency, never treats successful high-capacity telemetry as a certified counterfactual lower target, and uses observed headroom only to moderate the depth of measured probes. Its state and per-window control cost are $O(1)$; the only additional cost of exploration is the occasional window executed at the probed capacity.

6.3 Materiality–Cost Ambiguity Guard

The service-margin test controls certificate slack over the HH-relevant target set, but it does not isolate the special case in which an unresolved certificate crosses the heavy-hitter threshold. Such ambiguity is not solely an artifact of approximate counters. When a partition does not report key k , the coordinator knows only that its hidden contribution is bounded, not that it is zero. Threshold-crossing intervals may therefore remain even if every reported count is exact.

Not every unresolved key warrants additional memory. An estimate arbitrarily close to $1/n$ can require a very large capacity for strict classification while representing a negligible threshold distinction. The controller therefore balances two observable quantities: the material distinction resolved by a candidate capacity and the proportional memory increase required to obtain it.

For $k \in K_\gamma^t$, define the obstructing one-sided uncertainty and the point estimate's threshold distance by

$$(U_A^t(k), \Delta_A^t(k)) := \begin{cases} (\hat{p}^t(k) - \underline{p}^t(k), \hat{p}^t(k) - 1/n), & \hat{p}^t(k) \geq 1/n, \\ (\bar{p}^t(k) - \hat{p}^t(k), 1/n - \hat{p}^t(k)), & \hat{p}^t(k) < 1/n. \end{cases} \quad (18)$$

Under the same inverse-capacity approximation used for upward response, the smallest integer capacity above the observed operating point that strictly separates the estimated side of the interval is

$$s^t(k) := \max\left\{n, \left\lceil q^t \frac{U_A^t(k)}{\Delta_A^t(k)} \right\rceil + 1\right\}, \quad \Delta_A^t(k) > 0. \quad (19)$$

An exact estimated tie has $\Delta_A^t(k) = 0$ and is not assigned a finite separation demand. Instead, every finite demand receives the materiality weight

$$w_A^t(k) := n \Delta_A^t(k). \quad (20)$$

The weight measures distance from the decision threshold in threshold units. Consequently, an arbitrarily fine near-tie carries vanishing influence rather than an unbounded memory demand.

Let $\bar{q}^{t+1}$ denote the baseline target selected by the upward-response and probing controller. Let $\mathcal{K}_A^t := \{k \in K_A^t : s^t(k) > \bar{q}^{t+1}\}$ contain the unresolved keys whose finite separation demands are not already covered by that target, and let $C_A^t := \{s^t(k) : k \in \mathcal{K}_A^t\}$. For a candidate capacity $x \in C_A^t$, define the fraction of remaining material ambiguity resolved by

$$G_A^t(x) := \frac{\sum_{k \in \mathcal{K}_A^t} w_A^t(k) \mathbf{1}[s^t(k) \leq x]}{\sum_{k \in \mathcal{K}_A^t} w_A^t(k)} \quad (21)$$

and the proportional incremental cost by

$$C_A^t(x) := \frac{x - \bar{q}^{t+1}}{\bar{q}^{t+1}}. \quad (22)$$

The observable frontier utility is $V_A^t(x) := G_A^t(x) - C_A^t(x)$. The selected ambiguity proposal is

$$A^{t+1} := \begin{cases} \arg \max_{x \in C_A^t} V_A^t(x), & C_A^t \neq \emptyset \text{ and } \max_{x \in C_A^t} V_A^t(x) > 0, \\ \bar{q}^{t+1}, & \text{otherwise.} \end{cases} \quad (23)$$

Ties are resolved in favor of the smaller capacity. This rule introduces neither an ambiguity-specific gain nor an external capacity cap. Since $G_A^t(x) \leq 1$, positive utility implies $C_A^t(x) < 1$, hence every selected ambiguity proposal satisfies $A^{t+1} < 2\bar{q}^{t+1}$. The bound follows directly from the frontier objective.

The observed ambiguity lift is represented in log-capacity space:

$$\ell_{\text{Amb}}^t := \left\lceil \log \frac{A^{t+1}}{\bar{q}^{t+1}} \right\rceil_+. \quad (24)$$

Because ambiguity may persist across neighboring windows, the controller retains this signal independently:

$$a_{\text{Amb}}^t := \max\{\ell_{\text{Amb}}^t, \beta a_{\text{Amb}}^{t-1}\}, \quad a_{\text{Amb}}^{-1} := 0. \quad (25)$$

Here β has one role only: it is the fraction of an unrefreshed ambiguity margin retained for one additional window. The default implementation uses $\beta = 1/2$. Crucially, a_{Amb}^t remains independent of the baseline controller. Ambiguity cannot alter the failed-probe boundary L^t , the recovery guard g^t , or any other temporal-control state, and baseline updates cannot manufacture ambiguity.

6.4 Final Actuation

The final actuation combines the baseline target with the independently persisted ambiguity lift:

$$q^{t+1} := \lceil \bar{q}^{t+1} \exp(a_{\text{Amb}}^t) \rceil. \quad (26)$$

The baseline controller always enforces $\bar{q}^{t+1} \geq n$, and $a_{\text{Amb}}^t \geq 0$. Equation (26) therefore preserves the discoverability floor of Proposition 2 and can never lower the baseline. Recent threshold-crossing ambiguity can raise the next-window target, but only

through its own decayed log-capacity margin. It never alters the upward-response or probing state.

Equation (26) is the canonical control law of the framework. The temporal controller supplies $\bar{q}^{t+1}$; unresolved threshold-crossing certificates can only raise that value through their independently persisted ambiguity margin.

6.5 Interpretation of the Control Objective

The adaptive controller should be interpreted as a soft memory-risk controller. Its objective is not to guarantee that every future window will satisfy the deterministic margin target r_M for every HH-relevant candidate. Rather, it combines capacity-relative margin evidence, conservative recovery from failed downward probes, and observable evidence that additional memory can resolve materially relevant threshold ambiguity.

By construction, $q^{t+1} \geq \bar{q}^{t+1}$: ambiguity can never lower the baseline target. It raises memory only when a positive-utility frontier point resolves enough material ambiguity to justify its proportional cost, or while a previously observed ambiguity margin remains active under the decay rule.

This interpretation removes the need for an ambiguity-specific gain, resolution floor, or capacity cap. The frontier translates deterministic certificate geometry directly into a bounded memory decision, while β supplies short-lived persistence only to the ambiguity margin.

The adaptive sizing layer is now complete. The controller increases memory from observed margin violations, releases it only through physical probes, constructs an independent ambiguity proposal from the materiality-cost frontier, and takes the larger proposal at actuation. Baseline sufficiency evidence and certificate-completeness pressure therefore interact only at final actuation, not through each other's state.

This formulation preserves the main design principle of the paper: every control signal is expressed directly in memory units or as a log-capacity margin. The remaining section shows how the same certification and planning logic extends to a richer local summary regime with an exact head and an approximate residual tail.

7 HYBRID EXACT-APPROXIMATE TRACKING

Sections 2–6 developed the core framework under a deliberately simple regime: each partition maintains a single Space-Saving summary of common size q^t , the coordinator derives deterministic heavy-hitter certificates from the resulting telemetry, and the controller sizes the next window using one-sided upward pressure, residual-guarded downward probes, and a separate ambiguity guard. This common-size regime is sufficient to state the main certification and planning results cleanly. In practice, however, some workloads contain a small stable head of dominant keys whose repeated approximate tracking is wasteful. This section shows how to reserve exact state for such keys while preserving the same certificate-driven planning architecture.

When a small set of important keys persists across windows, it may be worthwhile to track those keys exactly while applying Space-Saving only to the residual tail. Importance can come from two sources. A key may be a stable dominant item whose mass

is repeatedly large, or it may lie on the unresolved heavy-hitter boundary, where exact tracking can collapse a threshold-crossing interval if the key persists. This yields a hybrid regime that separates selected head mass from the dynamically changing remainder of the stream.

Fix a coordinator-chosen promoted head $E^t \subseteq \mathcal{K}$ in window t . Partition i then maintains:

- (1) exact counters for the keys in E^t , and
- (2) a Space-Saving summary $\mathcal{S}_{i,\text{res}}^t$ of size r^t over the residual sub-stream

$$\mathcal{W}_{i,\text{res}}^t := \{r \in \mathcal{W}_i^t : \text{key}(r) \notin E^t\}.$$

Let

$$N_{i,\text{res}}^t := |\mathcal{W}_{i,\text{res}}^t|, \quad N_{\text{res}}^t := \sum_{i=1}^m N_{i,\text{res}}^t.$$

Let

$$\mathcal{A}_{\text{res}}^t := \{i \in [m] : N_{i,\text{res}}^t > 0\}$$

denote the residual-active partitions. Residual local relative frequencies are used only for partitions in $\mathcal{A}_{\text{res}}^t$; residual-inactive partitions contribute zero mass and, by convention below, zero minimum-counter terms. We also write

$$P_E^t := \sum_{x \in E^t} p^t(x) = 1 - \frac{N_{\text{res}}^t}{N^t}$$

for the global head mass captured exactly.

The coordinator-side candidate set in the hybrid regime becomes

$$\mathcal{U}_{\text{hyb}}^t := E^t \cup \bigcup_{i=1}^m \text{keys}(\mathcal{S}_{i,\text{res}}^t).$$

Keys in E^t are aggregated exactly:

$$\hat{f}^t(x) = f^t(x) = \sum_{i=1}^m f_i^t(x), \quad x \in E^t.$$

Residual keys $k \notin E^t$ are certified exactly as in Section 4, but now with respect to the residual summaries. Define the residual reporting set

$$R_{\text{res}}^t(k) := \{i \in [m] : k \in \text{keys}(\mathcal{S}_{i,\text{res}}^t)\},$$

the residual minimum counter

$$c_{i,\text{res}}^t := \begin{cases} 0, & |\mathcal{S}_{i,\text{res}}^t| < r^t, \\ \min_{x \in \text{keys}(\mathcal{S}_{i,\text{res}}^t)} \hat{f}_{i,\text{res}}^t(x), & |\mathcal{S}_{i,\text{res}}^t| = r^t, \end{cases}$$

again treating unused residual counters as zero, and the residual aggregate estimate

$$\hat{f}_{\text{res}}^t(k) := \sum_{i \in R_{\text{res}}^t(k)} \hat{f}_{i,\text{res}}^t(k).$$

Then the same signed decomposition as before yields

$$- \sum_{j \notin R_{\text{res}}^t(k)} c_{j,\text{res}}^t \leq \hat{f}_{\text{res}}^t(k) - f^t(k) \leq \sum_{i \in R_{\text{res}}^t(k)} \varepsilon_{i,\text{res}}^t(k),$$

and hence deterministic intervals for residual keys:

$$p_{\text{res}}^t(k) = \max \left\{ 0, \frac{\hat{f}_{\text{res}}^t(k)}{N^t} - \frac{1}{N^t} \sum_{i \in R_{\text{res}}^t(k)} \varepsilon_{i,\text{res}}^t(k) \right\}, \quad (27)$$

$$\bar{p}_{\text{res}}^t(k) = \min \left\{ 1, \frac{\hat{f}_{\text{res}}^t(k)}{N^t} + \frac{1}{N^t} \sum_{j \notin R_{\text{res}}^t(k)} c_{j,\text{res}}^t \right\}. \quad (28)$$

For head keys $x \in E^t$, the interval collapses to the point mass

$$\underline{p}^t(x) = \bar{p}^t(x) = p^t(x).$$

The hybrid regime also admits a residual discoverability guarantee.

PROPOSITION 5 (RESIDUAL DISCOVERABILITY FLOOR). *If $N_{\text{res}}^t = 0$, no residual key can be a global heavy hitter. Otherwise, define the integer residual floor*

$$r_{\text{min}}^t := \max\{1, \lceil n(1 - P_E^t) \rceil\}.$$

If

$$r^t \geq r_{\text{min}}^t,$$

then every global heavy hitter $k \notin E^t$ appears in the residual candidate set

$$\bigcup_{i=1}^m \text{keys}(\mathcal{S}_{i,\text{res}}^t).$$

PROOF. If $N_{\text{res}}^t = 0$, then every key outside E^t has zero frequency, so the claim is vacuous. Suppose $N_{\text{res}}^t > 0$, and let $k \notin E^t$ satisfy $p^t(k) > 1/n$. Since k is not in the exact head, all of its mass lies in the residual stream, so

$$f^t(k) = \sum_{i=1}^m f_i^t(k) \quad \text{and} \quad N_{\text{res}}^t = (1 - P_E^t)N^t.$$

Therefore

$$\frac{f^t(k)}{N_{\text{res}}^t} > \frac{1/n}{1 - P_E^t} = \frac{1}{n(1 - P_E^t)}.$$

Moreover,

$$\frac{f^t(k)}{N_{\text{res}}^t} = \sum_{i \in \mathcal{A}_{\text{res}}^t} \frac{N_{i,\text{res}}^t}{N_{\text{res}}^t} \cdot \frac{f_i^t(k)}{N_{i,\text{res}}^t}.$$

Thus there exists some residual-active partition $i \in \mathcal{A}_{\text{res}}^t$ such that

$$\frac{f_i^t(k)}{N_{i,\text{res}}^t} > \frac{1}{n(1 - P_E^t)}.$$

Since $r^t \geq r_{\text{min}}^t \geq n(1 - P_E^t)$, we have

$$\frac{1}{r^t} \leq \frac{1}{n(1 - P_E^t)},$$

so

$$\frac{f_i^t(k)}{N_{i,\text{res}}^t} > \frac{1}{r^t}.$$

Lemma 2, applied to the residual summary, implies

$$k \in \text{keys}(\mathcal{S}_{i,\text{res}}^t).$$

□

This residual floor shows the value of exact head promotion: once a substantial fraction of the mass is removed into E^t , the approximate discoverability burden on the tail decreases proportionally.

The planning layer extends naturally as well. Write $e^t := |E^t|$ for the exact-head counters and r^t for the residual-summary counters at each partition. The residual tail admits its own service margin $M_{\text{res},\alpha}^t$ and upward proposal $r_{\uparrow}^t$, obtained by applying Section 5

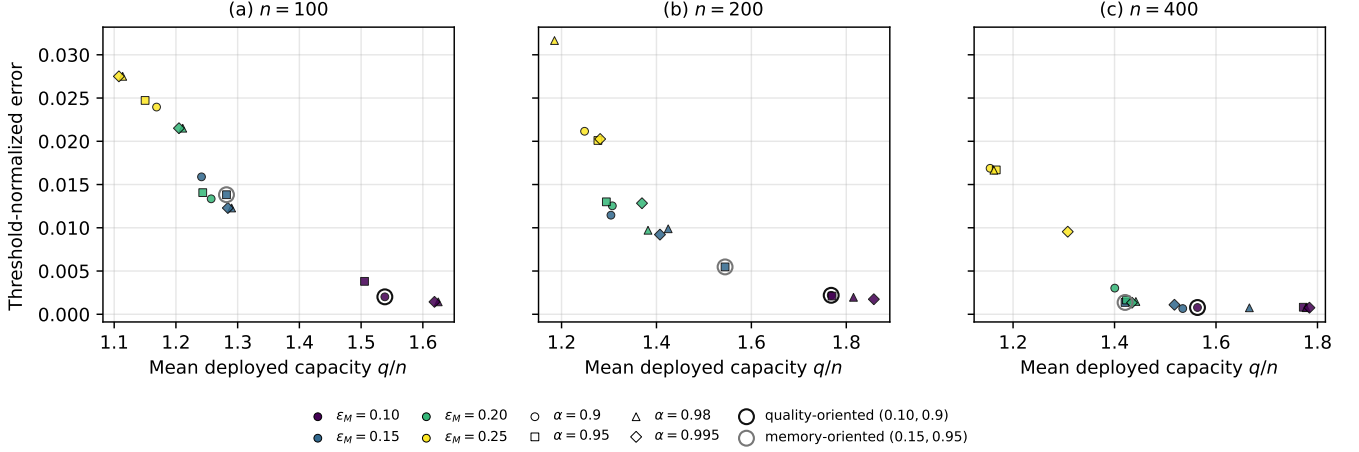


Figure 2: Margin and service-fraction calibration under the residual-guarded probing controller on MILP stress workloads with $m = 100$ and $n \in \{100, 200, 400\}$. Each point reports mean deployed capacity q/n and threshold-normalized error $n \cdot \text{AAE}/N$. The black ring marks the quality-oriented profile $(\epsilon_M, \alpha) = (0.10, 0.90)$; the gray ring marks the memory-oriented profile $(0.15, 0.95)$. All panels use the same vertical scale.

to the residual intervals and residual HH-relevant set. The total next-window local budget is then

$$b^{t+1} = e^{t+1} + r^{t+1},$$

where e^{t+1} is chosen by a head-promotion rule and r^{t+1} is produced by the upward-response and probing controller applied to the residual margin signal.

The choice of E^{t+1} is a promotion policy, not a new correctness assumption. It must be based on certificates and estimates available after window t , and it affects only which keys receive exact counters in the next window. We use the one-step promoted set

$$E^{t+1} = \text{Top}_n^t \cup K_+^t \cup K_-^t,$$

where Top_n^t denotes the n largest estimated candidates in the realized window. This promoted set captures stable high-rank keys even when certification is conservative, while still retaining all certified-positive and unresolved boundary keys. Longer-horizon variants can require repeated membership or rank stability before promotion, but the essential point is unchanged: the hybrid regime is a structured refinement of the same certification and capacity-pressure architecture, not a separate inference model.

The core framework of the paper remains intentionally narrow: a common-size local Space-Saving summary at each partition, deterministic coordinator-side certification, requirement inversion, and predictive next-window sizing. The hybrid regime broadens only the local telemetry contract: exact counters remove a stable head from the approximate problem, while residual summaries preserve the same signed-envelope analysis for the remaining keys.

This extension does not change the conceptual structure of the paper. Local telemetry still induces deterministic realized-window certificates, those certificates induce a margin test and one-sided pressure signal, and the controller combines upward response with measured release. The hybrid regime is therefore best understood

as a structured refinement of the local tracking layer rather than as an alternative formulation of the problem.

8 EXPERIMENTAL EVALUATION

8.1 Experimental Setup

We evaluate the framework in four stages. First, we calibrate the deterministic margin and service-fraction parameters used by the complete baseline controller. We then isolate the contribution of downward probing and failed-probe residual evidence, evaluate the ambiguity guard, and finally compare the resulting system against static internal and external baselines. The certification layer is also evaluated independently so that controller behavior can be separated from the correctness and sharpness of the underlying deterministic envelopes.

All experiments use a windowed stream distributed over m input partitions. The symbol n has the dual role established in Section 2: it is both the number of downstream workers and the denominator of the strict global heavy-hitter threshold $p(k) > 1/n$. Unless stated otherwise, each input partition maintains a common-size Space-Saving summary with per-item error telemetry, and the coordinator applies the deterministic reduction of Section 4. An exact oracle processes the same windows and supplies ground truth for evaluation only; it is not available to the controller.

Workloads and partitioning. The calibration workload is synthetic and intentionally difficult. Each window contains $N = 60,000$ records drawn from a drifting near-threshold normal profile, after which the global counts are distributed over $m = 100$ input partitions using the MILP stress transform of Appendix B.1. We instantiate the same workload family at $n \in \{100, 200, 400\}$, changing the global profile consistently with the corresponding $1/n$ threshold. This produces three topology settings rather than three unrelated tuning datasets.

The remaining evaluation will use scheduled-difficulty streams with easy, intermediate, and hard partitioning phases for the probing ablation; persistent-locality and MILP stress streams for ambiguity analysis; and benign partitioning together with real CAIDA-derived traces for the final robustness study. Each subsection states explicitly which workload is used. Synthetic stress workloads are not presented as models of ordinary traffic; they are controlled instruments for exercising certificate pressure and memory adaptation.

Methods and baselines. The core controller combines immediate upward response with physical downward probing. Its temporal ablation compares four controlled variants that share the same service margin and upward response but differ in probe selection and failed-probe protection. The ablation identifies guarded margin-comfort probing as the temporal policy used by the complete controller. The ambiguity study adds the ambiguity guard only after that policy has been fixed. Static Space-Saving provides the internal memory-quality reference, while HeavyLocker is retained as the external adversarial baseline under equal static local-memory budgets. The hybrid exact-approximate tracker is evaluated separately as an extension of the local telemetry layer.

Metrics. Heavy-hitter quality is reported through precision, recall, and F1 against the oracle set. Frequency quality is measured by average absolute error (AAE) on oracle heavy hitters. To compare different threshold resolutions, we report the threshold-normalized error

$$\frac{n \cdot \text{AAE}}{N},$$

which expresses the mean count error as a fraction of the heavy-hitter count threshold N/n . Memory is reported as the mean deployed capacity q/n , with worker memory in bytes used for equal-budget comparisons. Controller diagnostics include the observed service-violation rate $\Pr[M_\alpha^t > r_M]$, the number of downward probes, the fraction of probes that fail, and capacity volatility. Certification diagnostics include candidate recall, certified-positive precision, ambiguity mass and cardinality, and deterministic interval

width. Runtime experiments additionally report update throughput and coordinator reduction and control time.

Figure 4 exposes the corresponding memory trajectories. Margin-comfort probing releases memory in measured steps, while pressure gating often delays release after difficulty falls.

8.2 Controller Configuration

We configure the controller in stages. The first experiment selects two operating profiles for the margin and service-fraction pair under a fixed residual-guarded probing reference. The second isolates the temporal release mechanism and selects the final probe-depth policy. The third evaluates ambiguity actuation after that baseline controller has been fixed. This ordering separates the service objective from the choice of downward-release rule while retaining ablations that identify where the final behavior originates.

8.2.1 Margin and Service-Fraction Calibration. We first vary the fractional deterministic margin budget ϵ_M and service fraction α . As defined in Section 5.3, the absolute margin target is $r_M = \epsilon_M/n$. Hence ϵ_M specifies acceptable deterministic slack in units of the heavy-hitter threshold, while α determines the fraction of HH-relevant candidates represented by the window-level service margin M_α .

The sweep evaluates

$$\epsilon_M \in \{0.10, 0.15, 0.20, 0.25\}, \quad \alpha \in \{0.90, 0.95, 0.98, 0.995\}$$

for each $n \in \{100, 200, 400\}$. Every combination processes the same ordered windows from a fresh $q^0 = n$. The experiment fixes residual-guarded bracket probing throughout: upward response, downward probing after two sufficient observations, and failed-probe residual guarding are enabled. Holding the temporal mechanism fixed prevents the parameter sweep from conflating the service objective with probe-policy selection. Ambiguity adjustment is disabled.

Figure 2 shows the resulting memory-accuracy frontier. The shared vertical scale is deliberate: the $n = 400$ points occupy a narrow low-error band, whereas the smaller- n workloads distinguish the looser margin settings more sharply. Across all α values and

Table 2: Global-frequency scenarios used in the temporal-controller ablation. All windows have $N = 60,000$, 15,000 possible keys, and $n = 200$, so the heavy-hitter count threshold is $\tau = N/n = 300$. The stationary round-robin and MILP controls use the same generated global windows and differ only in their partitioning.

Experiment	Global-frequency profiles	Temporal program
Sharp step	E: 8 HHs with counts 1.8τ – 2.8τ , plus 7,000 uniformly weighted background keys. H: 65 near-threshold HHs at 1.005τ – 1.12τ , plus 14,900 uniform background keys.	$E^{30} \rightarrow H^{50} \rightarrow E^{30}$.
Smooth ramp	Same E and H profiles.	E^{15} , a 30-window interpolation to H, H^{10} , a 30-window interpolation back to E, then E^{15} .
Short bursts	Same E and H profiles.	$E^{24} \rightarrow H^3 \rightarrow E^{18} \rightarrow H^3 \rightarrow E^{18} \rightarrow H^3 \rightarrow E^{24}$.
Block oscillation	Same E and H profiles.	Four repetitions of $E^{12} \rightarrow H^{12}$.
Stationary controls	Normal mass profile over key rank, with 15,000 keys and $\sigma/K \in \{0.0052, 0.0050, 0.0052\}$. Its center moves through 0.20K, 0.45K, and 0.70K; HH creation is not forced.	30 windows at each center, separated by two 10-window interpolations. Per-window key evolution permits 35% hot-key drift, 25% relative-frequency change, and 12% new keys. The complete 110-window sequence is partitioned entirely by round robin in one control and entirely by MILP stress in the other.

topologies, the mean q/n decreases from 1.70 at $\varepsilon_M = 0.10$ to 1.41, 1.33, and 1.19 at 0.15, 0.20, and 0.25, respectively. The corresponding mean threshold-normalized errors are 0.00165, 0.00785, 0.01050, and 0.02138. Thus ε_M is the primary quality–memory control: 0.10 defines a visibly stricter accuracy regime, while 0.25 releases memory at a substantial quality cost.

The effect of α is secondary and is not monotone under closed-loop control. Changing the service quantile can alter an early upward response or probe outcome, which changes the capacity at which later telemetry is collected. Consequently, adjacent α values may follow different trajectories even though the service statistic itself is an ordered quantile. At the strict margin $\varepsilon_M = 0.10$, $\alpha = 0.90$ already produces uniformly low error and avoids paying for a more aggressive service fraction. At $\varepsilon_M = 0.15$, $\alpha = 0.95$ gives the lowest cross-topology mean error among the four service fractions without adopting the largest α .

We promote two operating choices. The pair (0.10, 0.90) is the *quality-oriented* profile: it applies a strict margin to 90% of the realized frontier and yields consistently low error without relying on an extreme service quantile. The *memory-oriented* profile (0.15, 0.95) lowers mean q/n by approximately 17%, 13%, and 9% for $n = 100, 200, 400$, respectively, while accepting higher error. The quality-oriented profile is the primary configuration in accuracy-sensitive comparisons; the memory-oriented profile is reported where the objective is to expose a more aggressive memory–quality tradeoff.

8.2.2 Temporal-Controller Ablation. We next isolate the temporal release mechanism using the quality-oriented profile $(\varepsilon_M, \alpha) = (0.10, 0.90)$, with ambiguity adjustment disabled. Every variant

starts from $q^0 = n$, applies the same origin-aware upward response when $M_\alpha^t > r_M$, and requires two consecutive sufficient observations before attempting a downward probe. The variants differ only in how the probe is selected and how failed-probe evidence is retained.

The ablation compares four variants. *Bracket probing* uses the geometric midpoint of its observed capacity bracket and discards failed-probe evidence. *Guarded bracket probing* retains the upward demand exposed by a failed probe until recovery is confirmed. *Guarded margin comfort* keeps that guard but replaces bracket interpolation with the capped arithmetic probe of Equation (14). Finally, *pressure-gated comfort* also vetoes a probe when $\rho > 0.5$. The pressure gate is an ablation mechanism rather than part of the proposed controller.

Workloads. We organize the ablation into two groups with different purposes. The first contains four scheduled profiles that move between easy round-robin, intermediate MILP, and hard MILP partitioning: a sharp step, a smooth ramp, short hard bursts, and repeated block oscillation. These workloads test whether the controller can release and recover memory as difficulty changes over time. The second group contains two stationary controls generated from matched global windows: full round-robin partitioning and full MILP stress partitioning. This pair holds temporal structure fixed and isolates the effect of partitioning difficulty. Keeping the groups separate avoids treating stationary controls as additional temporal profiles or obscuring transition behavior through a six-workload average.

Figure 3 shows why aggregate means alone are insufficient. Unguarded bracket probing repeatedly revisits capacities already



Figure 3: Per-window threshold-normalized error over four scheduled difficulty profiles. The dashed line is the configured margin budget $\varepsilon_M = 0.10$; background bands identify round-robin, intermediate-MILP, and MILP-stress phases.

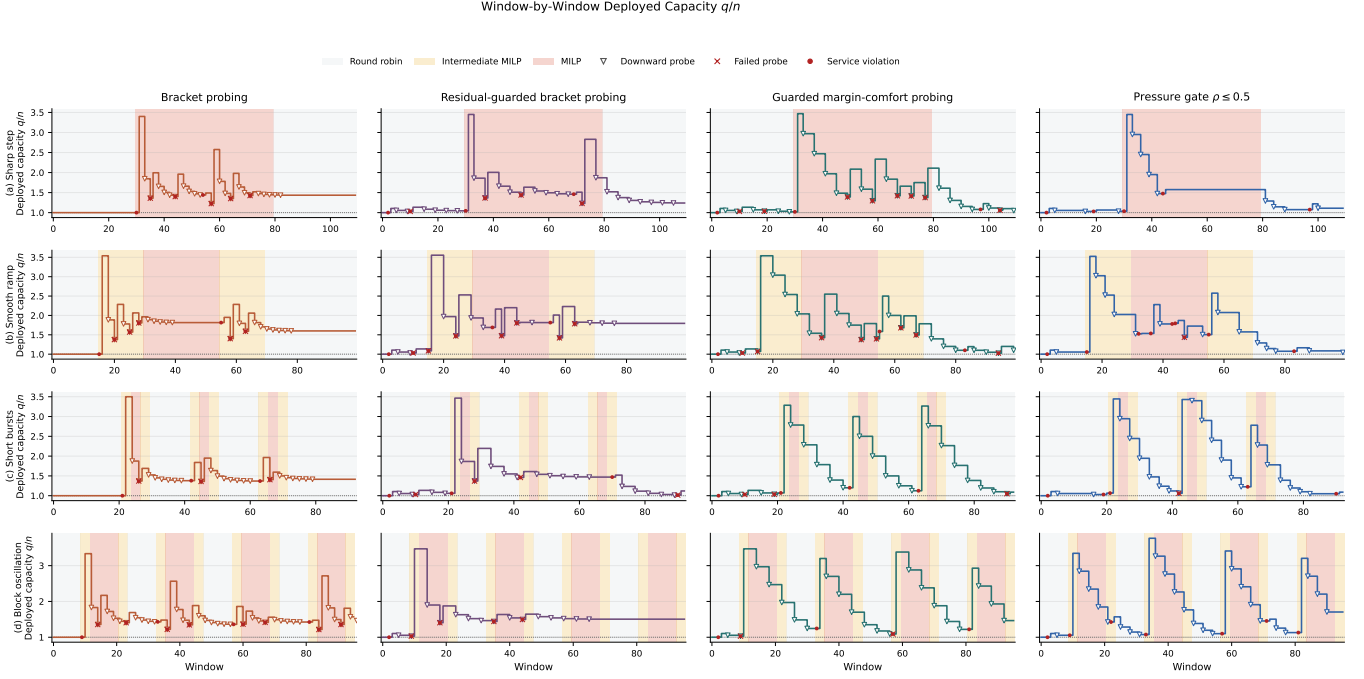


Figure 4: Per-window deployed capacity q/n over four scheduled difficulty profiles. Markers identify downward probes, failed probes, and service violations; background bands identify the active partitioning regime.

shown to be unsafe, producing abrupt error spikes after downward steps. Retaining failed-probe evidence removes many of these repetitions. Margin-comfort probing releases memory more gradually and tracks abrupt, bursty, and oscillating changes with fewer severe excursions, although its continued exploration is less advantageous on the smooth ramp. Pressure gating suppresses some probes, but can remain overprovisioned after difficulty falls. The relevant distinction is therefore not simply how often a policy probes, but whether its probe depth and retained failure evidence prevent the same unsafe reduction from recurring.

Figures 5 and 6 separate two persistent regimes. Under round robin, every policy periodically probes because high-capacity sufficiency cannot certify an unobserved lower capacity. Residual guarding and margin comfort reduce the resulting error excursions without freezing memory permanently. Under sustained MILP stress, failed reductions expose a recurring lower boundary. The guarded policies retain that evidence, whereas unguarded bracket probing repeatedly reconstructs the same sawtooth cycle. Pressure gating is the most reluctant to release memory, but its larger capacity does not consistently produce lower error than guarded margin comfort.

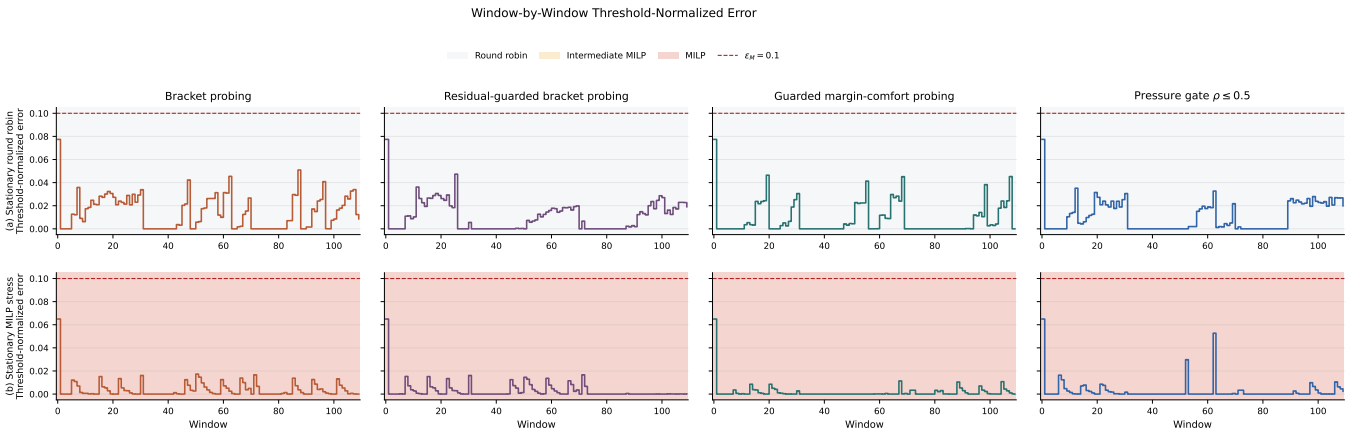


Figure 5: Per-window threshold-normalized error under matched stationary round-robin and MILP-stress partitioning. The dashed line is the configured margin budget $\epsilon_M = 0.10$.

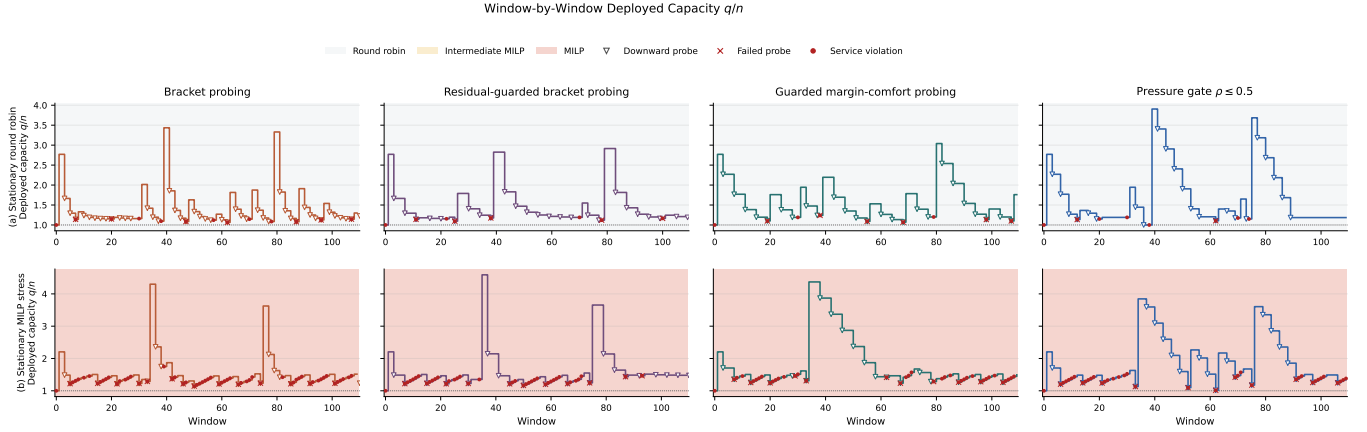


Figure 6: Per-window deployed capacity q/n under matched stationary round-robin and MILP-stress partitioning. Markers identify downward probes, failed probes, and service violations.

These results support guarded margin-comfort probing as the default temporal policy. The bracket variants remain diagnostic baselines, and pressure gating demonstrates that making downward release uniformly more conservative can suppress failures without improving the final tradeoff. The selected controller instead uses pressure to limit probe depth while continuing to validate every lower capacity physically.

8.2.3 Ambiguity-Guard Ablation. We finally isolate the ambiguity adjustment of Section 6.3. Both variants use the quality-oriented profile $(\epsilon_M, \alpha) = (0.10, 0.90)$ and guarded margin-comfort probing; they differ only in whether the materiality-cost ambiguity frontier is allowed to raise the baseline target. We evaluate the pair on the MILP stress workloads from Section 8.2.1 and on matched persistent-locality ambiguity-stress workloads. The latter preserve stable per-key partition affinities across windows, as described in Appendix B.2. Each experiment contains 110 windows over $m = 100$ input partitions. The ambiguity margin uses the fixed persistence factor $\beta = 0.5$.

Table 3 should be read as an actionability test for threshold-crossing uncertainty. On the MILP stress workloads, unresolved certificates are not merely present; they remain wide enough that extra local memory changes the downstream decision quality. The guard therefore converts a large fraction of ambiguous candidates into sharper certificates, and this is reflected in both lower threshold-normalized error and higher HH F1. The effect is strongest at $n = 100$ and $n = 200$, where the baseline controller still leaves substantial certificate geometry unresolved. At $n = 400$, the baseline already operates at a higher normalized capacity, so the same ambiguity signal has less room to improve quality and induces only a modest additional memory cost.

The persistent-locality workloads expose the other side of the design. They also contain repeated threshold-crossing ambiguity, but the baseline frequency error is already very small. In that regime, percentage error reductions can look large while the absolute improvement is less important. The more useful observation is that the guard does not behave like a fixed surcharge: as the baseline becomes sufficient, the selected lift contracts sharply, from a visible

premium at $n = 100$ to almost no added capacity at $n = 400$. In all cases the unresolved candidate set shrinks and F1 improves.

Actuation is frequent because the workloads deliberately maintain ambiguous near-threshold structure and because the ambiguity margin persists briefly across adjacent windows. Frequent actuation, however, is not equivalent to uncontrolled memory growth. The materiality-cost frontier raises capacity only when the unresolved certificates it can resolve justify the proportional increase over the temporal baseline. When ambiguity has largely been resolved already, the same rule does not keep spending memory arbitrarily; the remaining gain is limited, so the selected lift becomes marginal. The ablation therefore supports the guard as a conditional refinement of the complete controller: it is valuable when ambiguity is both observed and actionable, and its overhead contracts when the baseline already provides adequate resolution.

8.2.4 Selected Operating Profiles. The remainder of the evaluation carries the quality-oriented and memory-oriented margin profiles forward without retuning them per workload. Guarded margin-comfort probing is used as the temporal controller following Section 8.2.2; ambiguity is enabled only in experiments explicitly evaluating or using the complete controller.

8.3 Deterministic Certification Quality

This subsection will evaluate the realized-window certification layer independently of temporal control. The planned analysis varies local capacity and partitioning regime, then reports candidate recall, certified-positive precision, ambiguity mass, and interval width. This isolates the information quality available to any sizing policy built on the same telemetry.

8.4 End-to-End Adaptive Sizing

This subsection will evaluate the complete controller across stationary, drifting, scheduled-difficulty, adversarial, and real traces. Static Space-Saving capacities will define the memory-quality frontier against which the two promoted adaptive profiles are compared. The principal outputs will be threshold-normalized error,

HH quality, mean and volatile memory, and temporal response around regime changes.

8.5 Static-Memory Comparison with HeavyLocker

This subsection will compare against HeavyLocker under equal static local-memory budgets. Benign partitioning will establish the conventional baseline, while stress partitioning will test sensitivity to the random-routing assumption. The comparison will report HH precision, recall, F1, frequency error, and throughput without attributing adaptive behavior to a static baseline.

8.6 Hybrid Exact-Approximate Tracking

We next evaluate the hybrid extension of Section 7. The controller uses the quality-oriented profile from above, with $\epsilon_M = 0.10$, $\alpha = 0.90$, guarded margin-comfort probing, and ambiguity enabled. The only change is local telemetry: the baseline uses a single approximate Space-Saving summary, while the hybrid tracker promotes $\text{Top}_n^t \cup K_+^t \cup K_-^t$ from window t into exact counters for window $t + 1$, then applies the same controller to the residual tail.

Figure 7 shows that the hybrid layer is most useful when the promoted head removes mass that would otherwise consume approximate capacity. On the MILP stress workloads, hybrid tracking improves the memory and quality dimensions at the same time: threshold-normalized error falls by 13.6–54.2%, while worker memory falls by 18.0–25.2% for $n = 200$ and $n = 400$. This is the desired effect of the decomposition. Exact counters absorb keys that are persistently important or certificate-relevant, so the residual sketch spends its counters on a smaller and more informative tail.

The persistent-locality workloads are less demanding in absolute error, so the interpretation is different. For $n = 100$ and $n = 200$, the hybrid tracker still reduces both error and memory, but the absolute error levels are already near zero. At $n = 400$, the baseline has no measurable AAE left to reduce; the hybrid variant therefore buys a perfect HH F1 score at a small memory premium. This is not a reason to treat hybrid tracking as the default core mechanism. Rather, it supports the framing of Section 7: when stable or certificate-relevant head keys exist, exact promotion can sharpen the residual problem, but its value depends on whether the approximate baseline is still spending meaningful resources on that head.

Table 3: Ambiguity-guard ablation. The left group reports operation without the guard. The right group reports operation with the guard, followed by the change relative to the baseline in parentheses. Header arrows indicate the preferred direction for each metric. Error is $n \cdot \text{AAE}/N$, HH F1 is reported in percent, and Act. is the fraction of windows in which the ambiguity target exceeds the temporal baseline.

Workload	n	Guard off				Guard on (change)				
		Error ↓	$\bar{q}/n$ ↓	HH F1 (%) ↑	$ \bar{K}_? $ ↓	Error ↓	$\bar{q}/n$ ↓	HH F1 (%) ↑	$ \bar{K}_? $ ↓	Act.
MILP stress	100	4.07×10^{-3}	1.55	97.3	12.77	7.26×10^{-4} (−82.1%)	2.24 (+44.6%)	99.7 (+2.4)	1.58 (−87.6%)	100.0%
	200	1.73×10^{-3}	1.82	98.9	25.15	6.84×10^{-4} (−60.4%)	2.11 (+15.9%)	99.7 (+0.7)	3.80 (−84.9%)	100.0%
	400	9.71×10^{-4}	2.21	98.4	8.03	6.54×10^{-4} (−32.7%)	2.35 (+6.3%)	98.8 (+0.4)	7.36 (−8.3%)	100.0%
Persistent locality	100	1.92×10^{-5}	1.22	97.6	4.82	2.02×10^{-6} (−89.5%)	1.67 (+37.0%)	99.7 (+2.1)	0.61 (−87.4%)	100.0%
	200	1.47×10^{-5}	1.12	97.5	4.19	3.20×10^{-6} (−78.2%)	1.22 (+9.6%)	98.5 (+1.0)	2.45 (−41.4%)	97.3%
	400	1.44×10^{-6}	1.00	98.6	6.16	0 (−100.0%)	1.02 (+1.6%)	98.9 (+0.4)	4.70 (−23.7%)	94.5%

8.7 Runtime and Scalability

This subsection will report update throughput, coordinator reduction time, controller overhead, and telemetry memory while varying m , n , and candidate-set size. The goal is to separate the computational cost of deterministic certification from the cost of the local sketch updates.

8.8 Summary of Findings

This subsection will consolidate the completed findings after the remaining experiments are populated. It will distinguish empirical results about certificate quality, temporal adaptation, ambiguity actuation, external-baseline robustness, and runtime rather than repeating individual table entries.

9 RELATED WORK

Our framework sits at the intersection of streaming algorithms, distributed continuous monitoring, and adaptive resource provisioning. We review the most relevant literature in these areas and contrast it with our deterministic, capacity-pressure approach.

9.1 Fundamental Sketching Primitives

The foundation of modern stream processing relies on single-node sublinear data structures. Foundational algorithms such as Lossy Counting [16] and the Count-Min Sketch [7] provide robust mechanisms for frequency estimation and heavy-hitter tracking under bounded memory. Our local primitive of choice is Space-Saving, originally introduced via the Stream-Summary data structure by Metwally et al. [17].

Subsequent theoretical analysis by Cormode and Hadjieleftheriou [6] proved that counter-based methods like Space-Saving are remarkably space-efficient and provide strong approximation guarantees where the error depends only on the residual tail of the stream. Furthermore, recent systems like Memento [2] have explored utilizing Space-Saving over sliding windows to quickly and accurately detect emerging heavy hitters in network traffic. However, these algorithms and their optimizations are fundamentally designed for single-node visibility. Our work tackles the subsequent coordinator-side inference problem: how to construct certified global heavy-hitter bounds from these fragmented local views without assuming uniform routing or centralizing raw stream data.

Hybrid Exact-Approximate Tracking Ablation

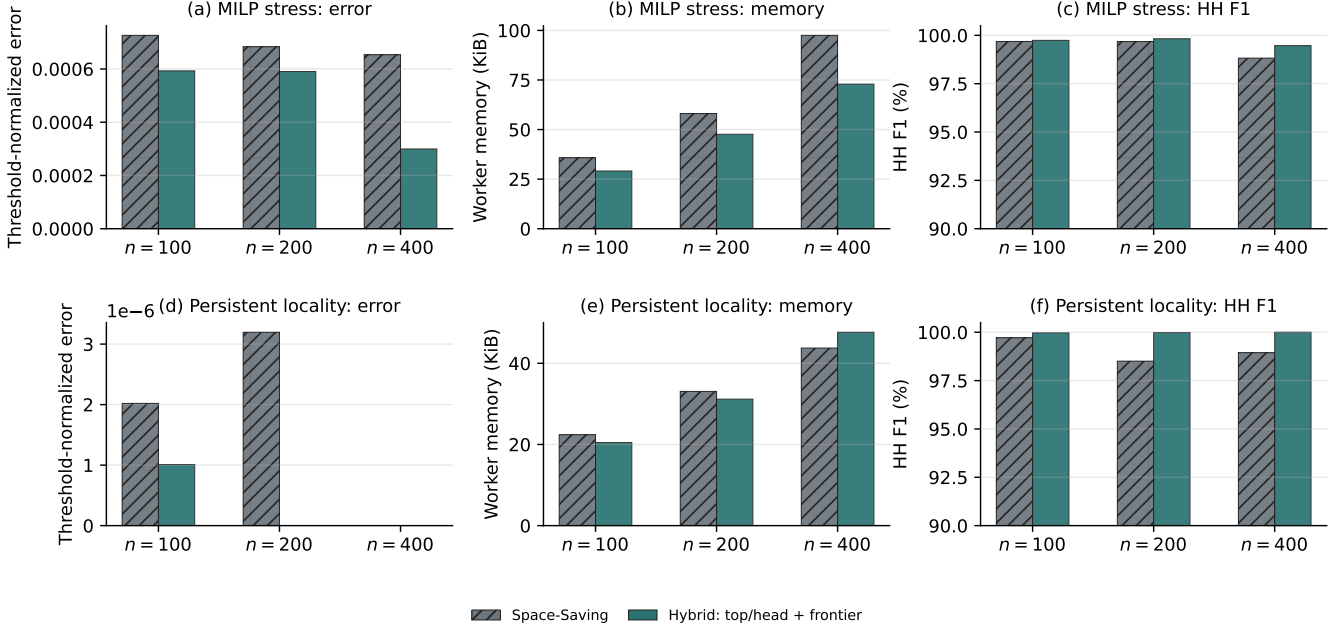


Figure 7: Hybrid exact-approximate tracking compared with the complete approximate controller. The hybrid variant promotes $\text{Top}_n^t \cup K_\dagger^t \cup K_\dagger^t$ into exact counters for the next window and sketches only the residual tail. Error is $n \cdot \text{AAE}/N$, memory is mean worker memory, and HH F1 is reported in percent.

9.2 Distributed and Continuous Monitoring

Extending single-node tracking to distributed topologies has been extensively studied, particularly in the context of continuous monitoring. The distributed streaming model formalizes scenarios where multiple streams are received at distributed sites that must communicate with a central coordinator to track global frequencies [18]. Seminal works by Babcock and Olston [1] on distributed top- k monitoring, alongside generalized distributed functional monitoring frameworks [5], tackle this by deploying local arithmetic constraints or threshold filters at the stream sources. In these systems, partitions only communicate with the coordinator when local constraints are violated, thereby minimizing network overhead.

Our setting operates under a distinctly different paradigm. Rather than optimizing continuous communication using local filters, we assume a fixed, windowed barrier telemetry contract where the primary bottleneck is bounded local memory. Furthermore, many distributed frameworks treat the hidden non-reporter mass problem probabilistically or rely on uniform hashing assumptions. By contrast, our framework constructs routing-agnostic, deterministic global envelopes from bounded telemetry.

9.3 Adaptive Sizing and Resource Provisioning

The predictive controller half of our architecture relates closely to dynamic resource provisioning and the vertical autoscaling of stateful operators in stream processors [10]. Systems like Dhalion

[9] and DS2 [11] introduce self-regulating topologies and automatic scaling policies to handle workload skew and throughput bottlenecks. These approaches typically monitor abstract, coarse-grained metrics—such as CPU utilization, backpressure, or estimated processing rates—and rely on heuristics or complex performance models to map those metrics into operator parallelism or memory allocations.

Our framework diverges from these methods by using deterministic certificate margins as capacity-relative control evidence. A margin violation is translated directly into an origin-aware upward demand, while successful telemetry authorizes no counterfactual decrease. Downward adaptation is instead performed through measured probes. This separation avoids treating a capacity-conditioned difficulty observation as a bidirectional memory label.

9.4 Deterministic vs. Probabilistic Guarantees

Streaming heavy-hitter algorithms occupy both randomized and deterministic parts of the design space. Hash-based sketches such as Count-Min provide compact frequency estimates with probabilistic error guarantees, typically expressed in terms of hash collisions and failure probability [7]. By contrast, counter-based summaries such as Lossy Counting and Space-Saving provide deterministic single-stream guarantees for L_1 -style frequency tracking [16, 17]. Our use of Space-Saving therefore does not claim determinism as a new property of the local primitive; rather, it uses that deterministic local

structure to build coordinator-side certificates from fragmented distributed views.

Deterministic guarantees also have known limits. For example, deterministic L_2 heavy-hitter sketches can require linear space in the ambient dimension [4]. Our setting is different: we certify L_1 relative-frequency heavy hitters from local counter-based telemetry at a window boundary. The core technical device—the signed decomposition of coordinator error into reporter inflation and hidden non-reporter mass—yields deterministic upper and lower bounds for the global L_1 relative frequencies of reported candidates. As a result, our certification sets (certified heavy hitters, non-heavy-hitters, and ambiguous keys) rely purely on the realized window, isolating the correctness guarantees of the certification layer from the success or failure of the predictive forecasting layer.

9.5 Temporal Correlation in Real Streams

The predictive layer of our framework is motivated by the observation that many real streams exhibit temporal correlation across adjacent windows. In such settings, the recent past often provides useful information about the near future, even when the workload remains highly nonstationary.

This phenomenon has been documented in data-center traffic. Benson et al. [3] report substantial temporal and spatial variation in production data-center traces and also find evidence of ON-OFF traffic behavior. Kandula et al. [12] similarly show that operational data-center traffic exhibits rich temporal structure and burstiness rather than simple memoryless behavior. More recently, Liu et al. [15] argue explicitly that *locality* is a key property of datacenter traffic demand and show strong dependency between spatial attributes and non-spatial attributes such as interarrival time and flow size. Taken together, these results support the use of short-horizon predictive mechanisms in systems that adapt resource allocation to recent traffic conditions.

A similar conclusion holds for textual and social-media streams. Diao et al. [8] show that microblog topics often evolve in bursty temporal patterns rather than as independent snapshots. Lehmann et al. [13] further demonstrate that hashtag popularity on Twitter follows structured attention trajectories, indicating that collective attention exhibits recognizable temporal classes. More recently, Liu et al. [14] study the persistence of news tweets and show that recent engagement behavior is predictive of short-term continuation. These results suggest that adjacent windows in textual streams often contain meaningful predictive signal about the continuation, decay, or transition of active topics.

Our predictive controller is therefore grounded in an empirical regularity that appears across multiple stream domains: recent windows are often informative about the next one. The controller does not require stationarity or exact repetition of past behavior. It only relies on the weaker premise that short-horizon temporal dependence is common enough in practice to make recent baseline requirements useful inputs for next-window provisioning, while ambiguity acts as a conservative certificate-completeness guard at actuation time.

A GLOBAL-FREQUENCY GENERATION

A.1 Controlled Heavy-Hitter/Background Distribution

The controlled heavy-hitter/background distribution, identified as `controlled_hh_background` in the generator configuration, is a per-window *global-frequency generator*, not a partitioning policy. It first determines the count of every key in a window; a separate policy then distributes those counts across the m input partitions. This separation allows the experiments to vary global frequency difficulty independently of local visibility.

For a window of mass N and heavy-hitter denominator n , let

$$\tau = \frac{N}{n}$$

denote the non-integer heavy-hitter count scale. The generator accepts a desired number h of heavy hitters and a ratio interval $[r_{\min}, r_{\max}]$, with $r_{\min} > 1$. It selects h designated keys and assigns linearly spaced ratios

$$r_1 = r_{\max} \geq r_2 \geq \dots \geq r_h = r_{\min}.$$

Their initial integer counts are

$$f(k_j) = \max\{\lfloor \tau \rfloor + 1, \text{round}(r_j \tau)\}, \quad j \in [h].$$

Thus each designated key is strictly globally heavy. If their requested total exceeds N , the implementation scales those counts down while retaining the strict floor $\lfloor \tau \rfloor + 1$.

The remaining mass is assigned to b background keys. Background weights are either uniform or linearly decreasing. For normalized weights w_j , the provisional background count is

$$f(u_j) = \min\left\{\lfloor \tau \rfloor - 1, \left\lfloor w_j \left(N - \sum_{\ell=1}^h f(k_\ell)\right) \right\rfloor\right\}.$$

The strict cap keeps every background key globally non-heavy. Integer remainders are distributed among background keys with available headroom. If the configured background population cannot absorb the complete remainder without crossing the cap, the residue is assigned round-robin to the designated heavy hitters. The final counts therefore satisfy

$$\sum_k f(k) = N,$$

while preserving the intended heavy/background classification.

This generator supports controlled regimes with either a small, clearly separated heavy head or many heavy hitters close to the decision boundary. For example, the temporal-controller workloads use an easy profile with eight keys at 1.8τ – 2.8τ , and a boundary-heavy profile with 65 keys at 1.005τ – 1.12τ . Transitions between such parameterizations alter the global distribution; the partitioning schedule independently controls how visible those global counts become at individual sites.

B IN-WINDOW PARTITIONING

Once a global-frequency generator has produced $f^t(k)$, an in-window partitioning policy assigns those counts to the m local streams. The following two policies preserve every global key count but induce different local evidence patterns.

B.1 MILP Stress Partitioning

This appendix specifies the adversarial partitioning transform used for the synthetic stress workloads in Section 8. The transform is applied after the global window distribution has already been generated. It therefore changes only the placement of records across input partitions, not the global key frequencies observed by the coordinator.

For a fixed window t , the global generator provides counts $f^t(k)$ satisfying $\sum_k f^t(k) = N^t$. A partitioning is admissible if it produces nonnegative integer local counts $f_i^t(k)$ such that

$$\sum_{i=1}^m f_i^t(k) = f^t(k) \quad \forall k,$$

and

$$\sum_k f_i^t(k) = N_i^t \quad \forall i,$$

where the target masses N_i^t satisfy $\sum_i N_i^t = N^t$. The MILP construction below is one admissible partitioning policy. Its purpose is not to change which keys are globally heavy, but to make the local evidence pattern as difficult as possible for bounded per-partition summaries.

B.1.1 Target Partition Masses. The target masses N_i^t are chosen before individual keys are placed. In the calibration workloads used in Section 8, we use balanced target masses up to integer rounding. This isolates the stress effect to the key placement rather than to gross partition-size imbalance. For each partition i , define the local strict reporting scale

$$T_i^t = \left\lceil \frac{N_i^t}{n} \right\rceil + 1.$$

A key whose local count reaches T_i^t is locally above the strict heavy-hitter count threshold induced by n , whereas counts below T_i^t remain locally sub-threshold.

B.1.2 Target Keys. Solving over every active key can be unnecessarily expensive, so the optimization focuses on the keys that can most affect certificate difficulty. Let $\mathcal{H}_n^t$ be the global heavy hitters from Definition 1. The implementation first computes a target budget

$$B^t = \min\{|\{k : f^t(k) > 0\}|, \max(B_0, B_{\min}, \lceil \gamma n \rceil)\},$$

where B_0 , $B_{\min}$, and γ are configuration parameters. In the calibration workloads, $B_0 = B_{\min} = 40$ and $\gamma = 0.4$, so the optimized target set contains at least forty keys when enough active keys exist and scales with the topology size through $\lceil 0.4n \rceil$.

The budget is split between true global heavy hitters and globally non-heavy distractors. A fraction ϕ_H of the budget is reserved for heavy hitters, with at least one heavy key selected when any exist. The selected heavy set $\mathcal{G}^t \subseteq \mathcal{H}_n^t$ prioritizes keys closest to the global threshold N^t/n , because these keys are hardest to certify from fragmented evidence.

The distractor pool contains globally non-heavy keys whose counts lie in a configurable near-threshold band

$$\rho_{\min} \frac{N^t}{n} \leq f^t(k) \leq \rho_{\max} \frac{N^t}{n}.$$

The deployed calibration uses $\rho_{\min} = 0.65$ and $\rho_{\max} = 0.995$. From this pool, the implementation selects keys closest to the global threshold, subject to the remaining target budget and the configured non-HH count multiplier. These keys form $\mathcal{D}^t$. They are globally non-heavy but can become locally salient in selected partitions. The optimized target set is

$$\mathcal{K}_\star^t = \mathcal{G}^t \cup \mathcal{D}^t.$$

If no eligible target keys exist, the implementation falls back to round-robin placement for that window. Otherwise, all keys outside $\mathcal{K}_\star^t$ are placed after the optimization as filler while preserving their global counts exactly.

B.1.3 MILP Formulation. For every target key $k \in \mathcal{K}_\star^t$ and partition i , the MILP uses an integer variable

$$x_i(k) \in \mathbb{Z}_{\geq 0}$$

for the local count assigned to partition i . It also uses a binary variable

$$z_i(k) \in \{0, 1\}$$

that indicates whether k is locally salient on partition i , and an auxiliary integer variable

$$u_i(k) \in \mathbb{Z}_{\geq 0}$$

that counts hidden sub-threshold mass for globally heavy target keys.

The first constraints preserve the global count of every optimized target key:

$$\sum_{i=1}^m x_i(k) = f^t(k) \quad \forall k \in \mathcal{K}_\star^t.$$

The optimized target keys may not exceed the target partition masses:

$$\sum_{k \in \mathcal{K}_\star^t} x_i(k) \leq N_i^t \quad \forall i.$$

To avoid degenerate solutions in which all mass of a key is concentrated in one partition, the solver bounds each $x_i(k)$ by a per-key concentration cap

$$x_i(k) \leq \beta f^t(k) \quad \forall i, k \in \mathcal{K}_\star^t,$$

for a fixed $\beta \in (0, 1]$, and also by the corresponding partition mass. The deployed calibration uses $\beta = 0.25$, so no optimized key may place more than one quarter of its global count on a single partition.

The binary salience variables are linked to the local threshold by standard big- M constraints:

$$x_i(k) \geq T_i^t z_i(k),$$

and

$$x_i(k) \leq T_i^t - 1 + f^t(k) z_i(k).$$

Thus $z_i(k) = 1$ forces the key to be locally above threshold, while $z_i(k) = 0$ forces it to remain below threshold on that partition.

For globally heavy target keys, the optimization rewards mass that is placed below local threshold. The auxiliary variables satisfy

$$u_i(k) \leq x_i(k),$$

and

$$u_i(k) \leq (T_i^t - 1)(1 - z_i(k)).$$

Consequently, $u_i(k)$ can be positive only when the key is locally sub-threshold on partition i , and it is bounded by the hidden local count assigned there.

The objective combines two adversarial goals:

$$\max \lambda_H \sum_{k \in \mathcal{G}^t} \sum_{i=1}^m u_i(k) + \lambda_D \sum_{k \in \mathcal{D}^t} \sum_{i=1}^m z_i(k),$$

where $\lambda_H, \lambda_D \geq 0$ weight the two effects. The first term hides as much globally heavy mass as possible below local thresholds. The second term makes globally non-heavy distractors locally salient, increasing competition inside bounded local summaries.

The live generator solves this MILP with integer $x_i(k)$, binary $z_i(k)$, and integer $u_i(k)$. The calibration configuration uses equal objective weights $\lambda_H = \lambda_D = 1$, a 300-second solver time limit, and a relative MIP gap tolerance of 0.01. If the solver returns a feasible incumbent, the resulting real-valued solution is rounded back to integer local counts with a deterministic repair pass that preserves each optimized key’s global count exactly.

B.1.4 Filler Placement. After the MILP places the target keys, all remaining keys are assigned deterministically to the residual partition capacities. The filler pass processes the remaining keys in decreasing global count order and repeatedly allocates each key across partitions with available residual capacity, using a stable hash only to break ties. This step preserves every global key count exactly and fills each partition to its prescribed mass. It does not optimize the adversarial objective, but it ensures that the final output is a complete admissible partitioning of the original generated window.

B.1.5 Properties of the Construction. The construction preserves the global workload exactly: for every key k , the coordinator-facing global count remains $f^t(k)$, and each partition has the prescribed mass N_i^t . It affects only the local evidence pattern seen by the summaries S_i^t .

The objective directly targets the two mechanisms that make coordinator-side inference difficult. First, a global heavy hitter can be made hard to certify by spreading its mass so that many partitions contain sub-threshold fragments. Second, globally non-heavy keys can be made misleading by concentrating enough mass on selected partitions to become locally salient without crossing the global heavy-hitter threshold. The resulting windows therefore contain both hidden heavy-hitter evidence and distracting local evidence.

The MILP is used only as a data-generation tool. The deterministic guarantees in Sections 3–4 do not depend on this construction. They apply to any admissible partitioning, including random, balanced, imbalanced, heuristic stress-partitioned, or MILP-generated windows. The transform is used only to generate synthetic evaluation cases in which the hidden non-reporter mass and misleading-local-evidence terms are operationally significant.

B.2 Persistent-Locality Partitioning

Persistent-locality partitioning models stable substream communities. Unlike the MILP transform, it does not optimize an adversarial objective. Instead, it assigns each key a deterministic affinity profile

over partitions and reuses that profile in every window. Consequently, a key may remain locally important to the same region, tenant, link, or shard even while its global count evolves.

Each key k is assigned, by a stable hash of the generator seed and key identifier, to one of four locality classes:

global, regional, local, background.

The configured fractions g, r, ℓ determine the first three class probabilities; any remaining probability is assigned to the background class. Regional and local keys receive deterministic home sets $H(k) \subseteq [m]$, containing configurable fractions of the partitions. Global and background keys use all partitions as their home set.

Let $s_k \in [0, 1]$ be the affinity strength of key k ’s class. Before normalization and optional bounded jitter, its partition weight is

$$w_i(k) = \frac{1 - s_k}{m} + \mathbf{1}[i \in H(k)] \frac{s_k}{|H(k)|}.$$

Thus $s_k = 0$ gives uniform placement, whereas larger values place more mass on the stable home set without excluding other partitions. A small deterministic multiplicative jitter can break otherwise identical profiles.

The implementation may first reserve a fraction $u \in [0, 1]$ of each key’s count as a uniform floor. For global count $f^t(k)$, it distributes $\text{round}(uf^t(k))$ approximately uniformly and allocates the remaining count according to the normalized affinity weights. Integer allocation uses a deterministic largest-remainder procedure, yielding local counts satisfying

$$\sum_{i=1}^m f_i^t(k) = f^t(k) \quad \forall k, t.$$

The key’s class, home set, affinity strength, and jitter seed depend on (k, seed) , but not on the window index. This is the source of temporal persistence: only $f^t(k)$ changes between windows, while the relative geography of that mass remains stable. The construction therefore preserves every generated global distribution exactly and introduces no assumption that partition-local streams are independent or randomly routed. It is used as a synthetic model of workloads with durable spatial affinity, such as geographically localized topics, tenant-to-shard mappings, recurring network prefixes, and stable service or cache communities.

ACKNOWLEDGMENTS

This work was supported by the [...] Research Fund of [...] (Number [...]). Additional funding was provided by [...] and [...]. We also thank [...] for contributing [...].

REFERENCES

- [1] Brian Babcock and Chris Olston. 2003. Distributed top-k monitoring. In *Proceedings of the 2003 ACM SIGMOD international conference on Management of data*. 28–39.
- [2] Ran Belhassen et al. 2018. Memento: Making sliding windows efficient for heavy hitters. In *Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies*. 250–262.
- [3] Theophilus Benson, Ashok Anand, Aditya Akella, and Ming Zhang. 2009. Understanding Data Center Traffic Characteristics. In *Proceedings of the ACM SIGCOMM Workshop on Research on Enterprise Networking (WREN)*. ACM, 65–72. <https://doi.org/10.1145/1672308.1672325>
- [4] Vladimir Braverman et al. 2018. Deterministic Heavy Hitters with Sublinear Query Time. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2018)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.

- [5] Graham Cormode. 2013. Continuous distributed monitoring: A short survey. *Transactions on Computational Collective Intelligence X* (2013), 24–39.
- [6] Graham Cormode and Marios Hadjieleftheriou. 2010. Space-optimal Heavy Hitters with Strong Error Bounds. *ACM Transactions on Database Systems (TODS)* (2010).
- [7] Graham Cormode and S Muthukrishnan. 2005. An improved data stream summary: the count-min sketch and its applications. *Journal of Algorithms* 55, 1 (2005), 58–75.
- [8] Qiming Diao, Jing Jiang, Feida Zhu, and Ee-Peng Lim. 2012. Finding Bursty Topics from Microblogs. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Jeju Island, Korea, 536–544. <https://aclanthology.org/P12-1056/>
- [9] Avriela Floratou, Ashvin Agrawal, Bill Graham, Sriram Rao, and Karthik Ramasamy. 2017. Dhalion: self-regulating stream processing in heron. *Proceedings of the VLDB Endowment* 10, 12 (2017), 1825–1836.
- [10] Vasiliki Kalavri et al. 2020. Performance Modeling and Vertical Autoscaling of Stream Joins. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*.
- [11] Vasiliki Kalavri, John Li, Alexandros Vishwanath, Stephen Shirer, Margo Vora, et al. 2018. Three steps is all you need: fast, accurate, automatic scaling decisions for distributed streaming dataflows. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 783–798.
- [12] Srikanth Kandula, Sudipta Sengupta, Albert G. Greenberg, Parveen Patel, and Ronnie Chaiken. 2009. The Nature of Datacenter Traffic: Measurements & Analysis. In *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement (IMC)*. ACM, 202–208. <https://doi.org/10.1145/1644893.1644918>
- [13] Janette Lehmann, Bruno Gonçalves, José J. Ramasco, and Ciro Cattuto. 2012. Dynamical Classes of Collective Attention in Twitter. In *Proceedings of the 21st International Conference on World Wide Web (WWW)*. ACM, 251–260. <https://doi.org/10.1145/2187836.2187871>
- [14] Maggie Liu, Jing Wang, and Daniel Preotiuc-Pietro. 2023. Analyzing and Predicting Persistence of News Tweets. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Nusa Dua, Bali, 719–732. <https://doi.org/10.18653/v1/2023.ijcnlp-main.46>
- [15] Zhiwen Liu, Mowei Wang, and Yong Cui. 2022. Locality Matters! Traffic Demand Modeling in Datacenter Networks. In *Proceedings of the 6th Asia-Pacific Workshop on Networking (APNet)*. ACM, 8–13. <https://doi.org/10.1145/3542637.3542639>
- [16] Gurmeet Singh Manku and Rajeev Motwani. 2002. Approximate frequency counts over data streams. In *Proceedings of the 28th international conference on Very Large Data Bases*. 346–357.
- [17] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. 2005. Efficient computation of frequent and top-k elements in data streams. In *International Conference on Database Theory*. Springer, 398–412.
- [18] Qin Zhang et al. 2013. Optimal tracking of distributed heavy hitters and quantiles. In *Proceedings of the 32nd ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 225–236.